

(12) **United States Patent**
Abe et al.

(10) **Patent No.:** **US 12,389,015 B2**
(45) **Date of Patent:** ***Aug. 12, 2025**

(54) **ENCODER, DECODER, ENCODING METHOD, AND DECODING METHOD**

(71) Applicant: **Panasonic Intellectual Property Corporation of America**, Torrance, CA (US)

(72) Inventors: **Kiyofumi Abe**, Osaka (JP); **Takahiro Nishi**, Nara (JP); **Tadamasa Toma**, Osaka (JP); **Ryuichi Kanoh**, Osaka (JP); **Chong Soon Lim**, Singapore (SG); **Ru Ling Liao**, Singapore (SG); **Hai Wei Sun**, Singapore (SG); **Sughosh Pavan Shashidhar**, Singapore (SG); **Han Boon Teo**, Singapore (SG); **Jing Ya Li**, Singapore (SG)

(73) Assignee: **PANASONIC INTELLECTUAL PROPERTY CORPORATION OF AMERICA**, Torrance, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/670,936**

(22) Filed: **May 22, 2024**

(65) **Prior Publication Data**
US 2024/0323405 A1 Sep. 26, 2024

Related U.S. Application Data
(63) Continuation of application No. 17/539,348, filed on Dec. 1, 2021, now Pat. No. 12,034,940, which is a (Continued)

(51) **Int. Cl.**
H04N 19/176 (2014.01)
H04N 19/105 (2014.01)
(Continued)

(52) **U.S. Cl.**
CPC **H04N 19/176** (2014.11); **H04N 19/105** (2014.11); **H04N 19/52** (2014.11); **H04N 19/70** (2014.11)

(58) **Field of Classification Search**
None
See application file for complete search history.

(56) **References Cited**
U.S. PATENT DOCUMENTS

6,473,458 B1 10/2002 Shimizu et al.
2014/0037011 A1 2/2014 Lim et al.
(Continued)

FOREIGN PATENT DOCUMENTS

JP 11-88890 3/1999

OTHER PUBLICATIONS

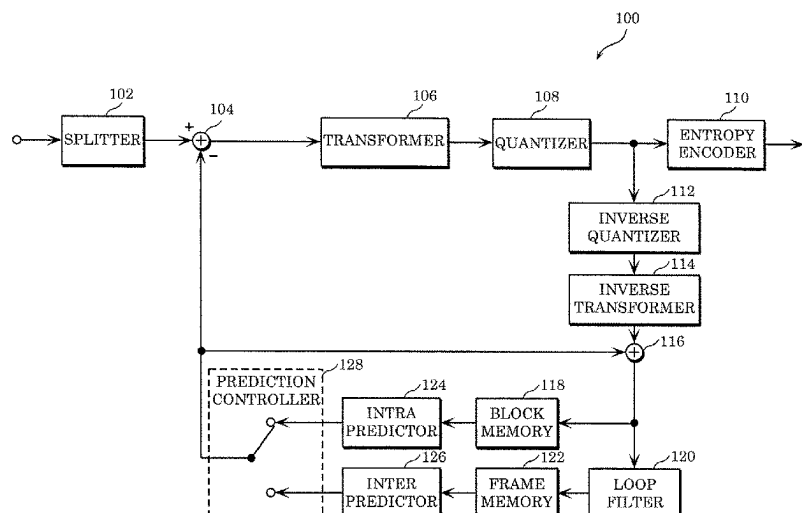
International Search Report issued Jul. 9, 2019 in International (PCT) Application No. PCT/JP2019/016574.
(Continued)

Primary Examiner — Christopher Braniff
(74) *Attorney, Agent, or Firm* — Wenderoth, Lind & Ponack, L.L.P.

(57) **ABSTRACT**

An encoder, includes: circuitry; and memory. Using the memory, the circuitry: in inter prediction for a current block, determines a base motion vector, and writes, in an encoded signal, a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and encodes the current block using the delta motion vector and the base motion vector as a motion vector of the current block.

2 Claims, 37 Drawing Sheets



Related U.S. Application Data

continuation of application No. 17/077,450, filed on Oct. 22, 2020, now Pat. No. 11,223,835, which is a continuation of application No. PCT/JP2019/016574, filed on Apr. 18, 2019.

- (60) Provisional application No. 62/662,524, filed on Apr. 25, 2018.

(51) **Int. Cl.**

H04N 19/52 (2014.01)

H04N 19/70 (2014.01)

(56) **References Cited**

U.S. PATENT DOCUMENTS

2017/0019671 A1 1/2017 Lim et al.
2017/0201751 A1 7/2017 Seo

2018/0041768 A1 2/2018 Koo et al.
2019/0182504 A1 6/2019 Lainema

OTHER PUBLICATIONS

H.265 (ISO/IEC 23008-2 High Efficiency video coding (HEVC)), Dec. 1, 2013.

Sri Nitchith Akula et al., "Description of SDR, HDR and 360° video coding technology proposal considering mobile application scenario by Samsung, Huawei, GoPro, and HiSilicon", buJoint Video Exploration Team (JVET) of ITU-T SG16 WP3 and ISO/IEC JTC1/SC29/WG11, Document: JVET-J0024_v2, Apr. 2018, pp. 4, 5, 9-13, 22, 23, 53, 54.

Li Jingya, et al., "CE4-related: Improvement on ultimate motion vector expression", Joint Video Exploration Team (JVET) of ITU-T SG16 WP3 and ISO/IEC JTC1/SC29/WG11, Document: JVET-L0408-v2, Oct. 2018, pp. 1-4.

Office Action issued Aug. 16, 2022 in Taiwanese Patent Application No. 108114145, with English-language translation of the Search Report.

FIG. 1

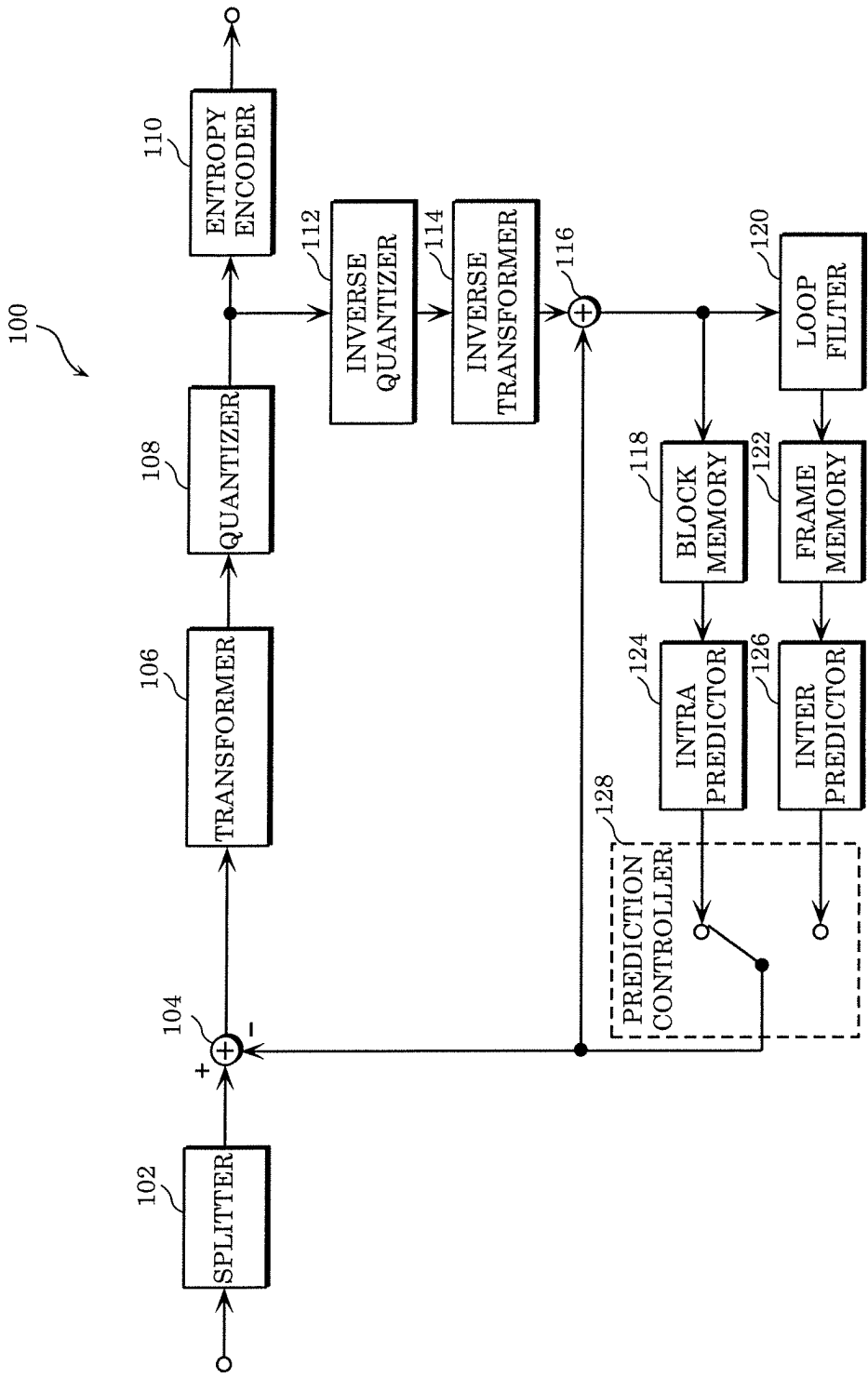


FIG. 2

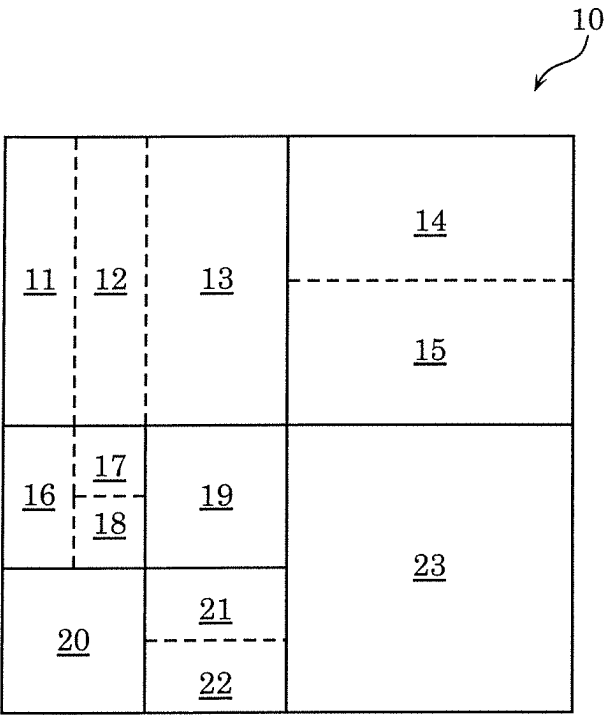


FIG. 3

TRANSFORM TYPE	BASIS FUNCTION $T_i(j)$, $i, j = 0, 1, \dots, N - 1$
DCT - II	$T_i(j) = \omega_0 \cdot \sqrt{\frac{2}{N}} \cdot \cos \left(\frac{\pi \cdot i \cdot (2j + 1)}{2N} \right)$ $\text{WHERE } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i = 0 \\ 1 & i \neq 0 \end{cases}$
DCT - V	$T_i(j) = \omega_0 \cdot \omega_1 \cdot \sqrt{\frac{2}{2N - 1}} \cdot \cos \left(\frac{2\pi \cdot i \cdot j}{2N - 1} \right)$ $\text{WHERE } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i = 0 \\ 1 & i \neq 0 \end{cases}, \omega_1 = \begin{cases} \sqrt{\frac{2}{N}} & j = 0 \\ 1 & j \neq 0 \end{cases}$
DCT - VIII	$T_i(j) = \sqrt{\frac{4}{2N + 1}} \cdot \cos \left(\frac{\pi \cdot (2i + 1) \cdot (2j + 1)}{4N + 2} \right)$
DST - I	$T_i(j) = \sqrt{\frac{2}{N + 1}} \cdot \sin \left(\frac{\pi \cdot (i + 1) \cdot (j + 1)}{N + 1} \right)$
DST - VII	$T_i(j) = \sqrt{\frac{4}{2N + 1}} \cdot \sin \left(\frac{\pi \cdot (2i + 1) \cdot (j + 1)}{2N + 1} \right)$

FIG. 4A

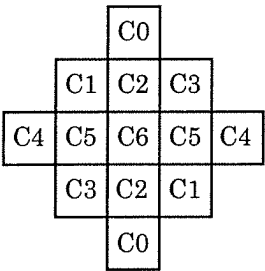


FIG. 4B

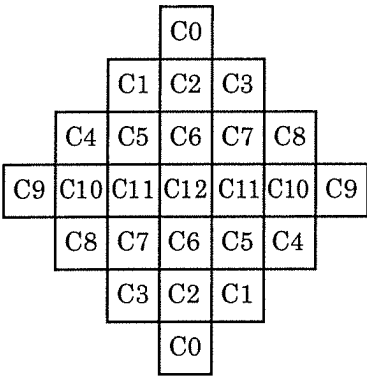


FIG. 4C

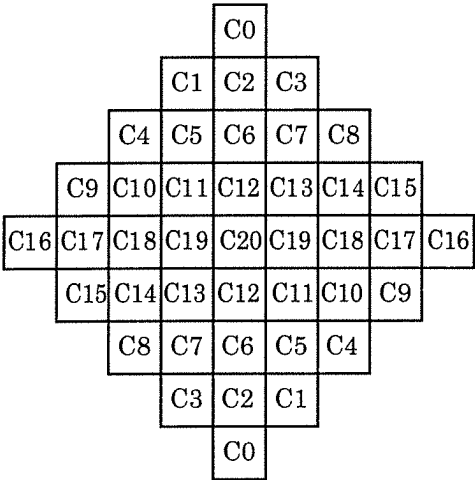


FIG. 5A

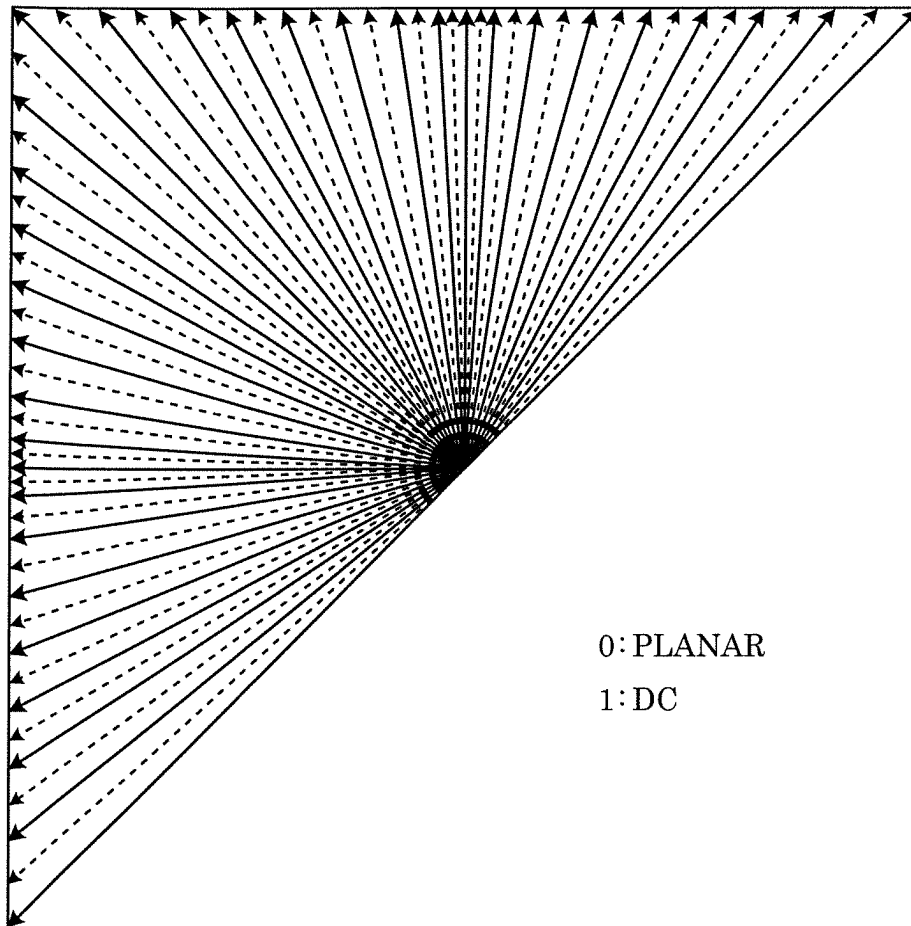


FIG. 5B

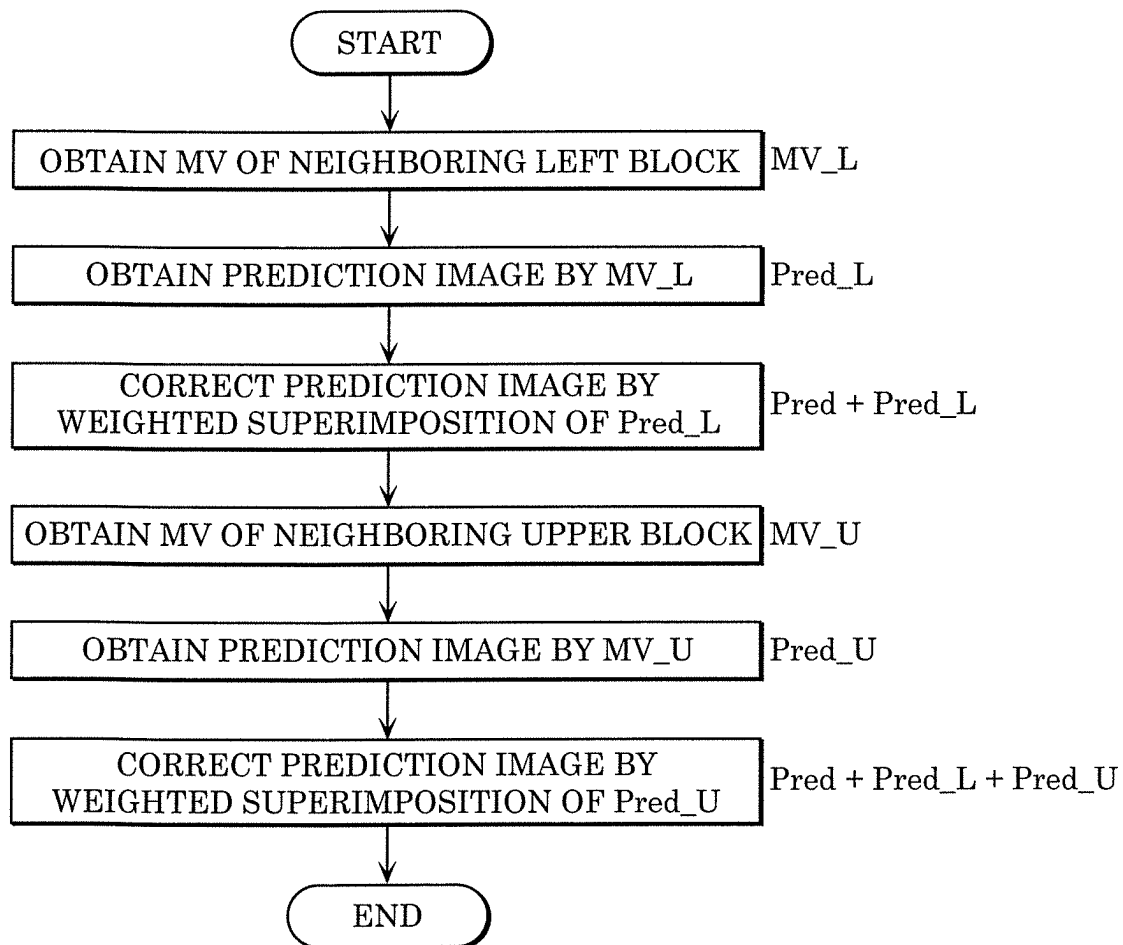


FIG. 5C

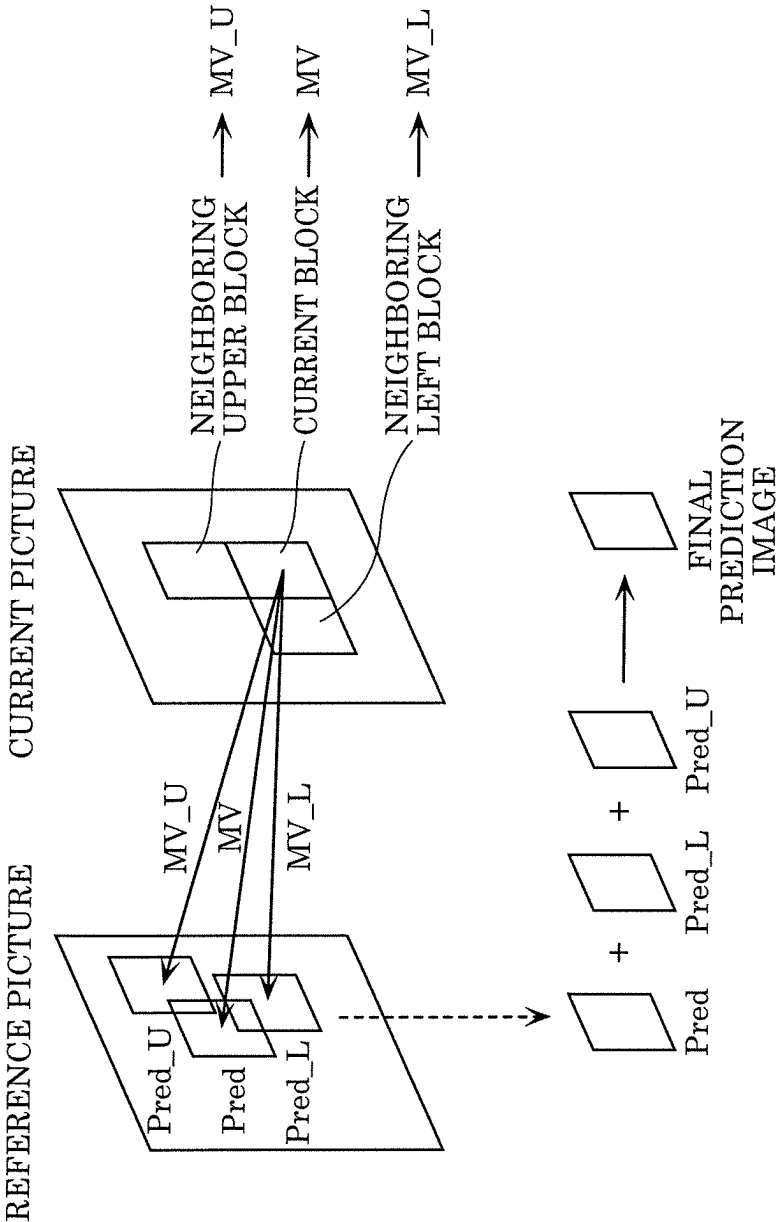


FIG. 5D

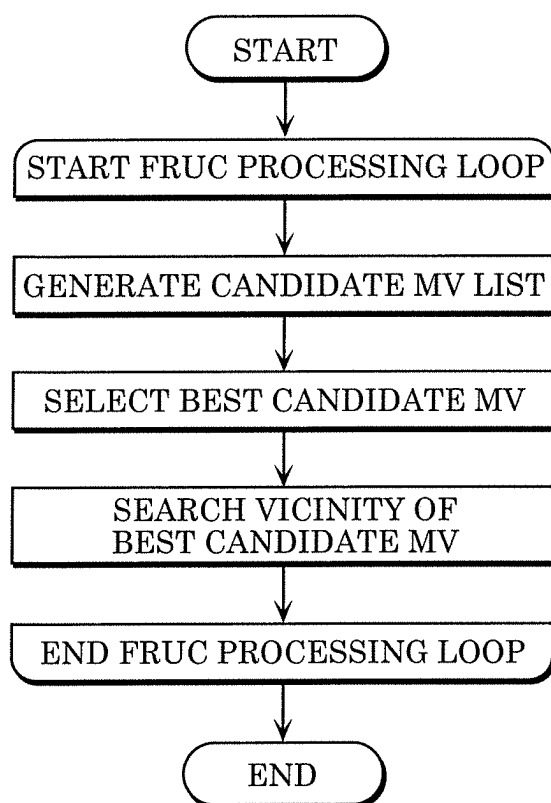


FIG. 6

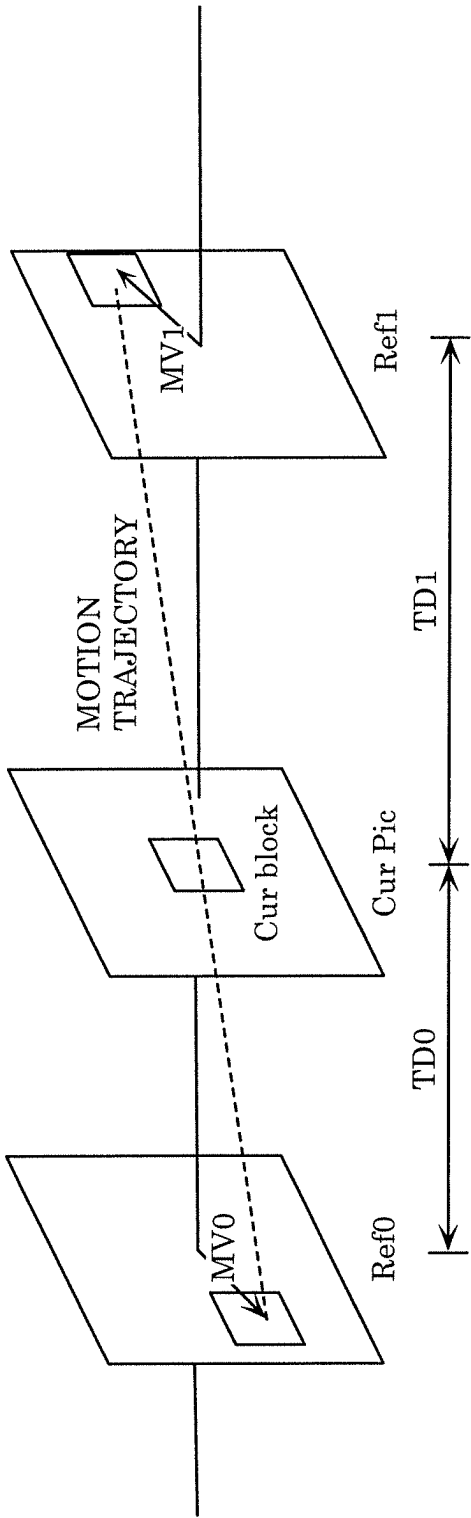


FIG. 7

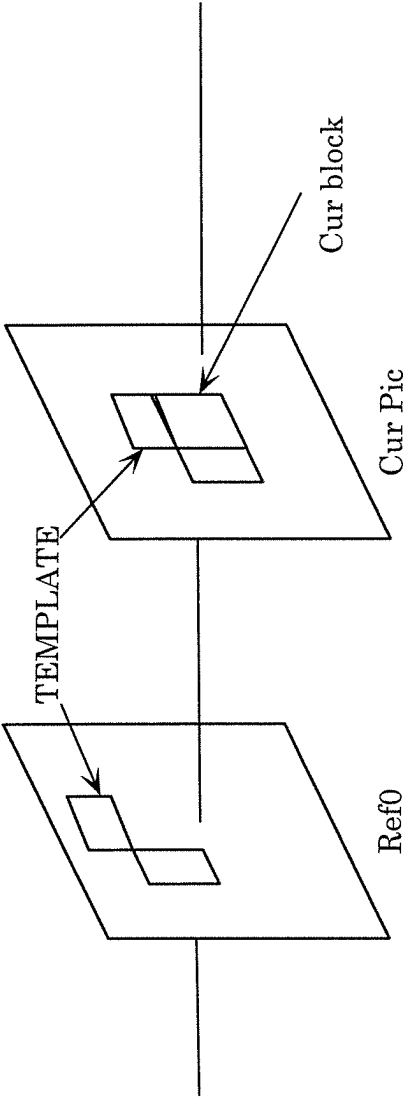


FIG. 8

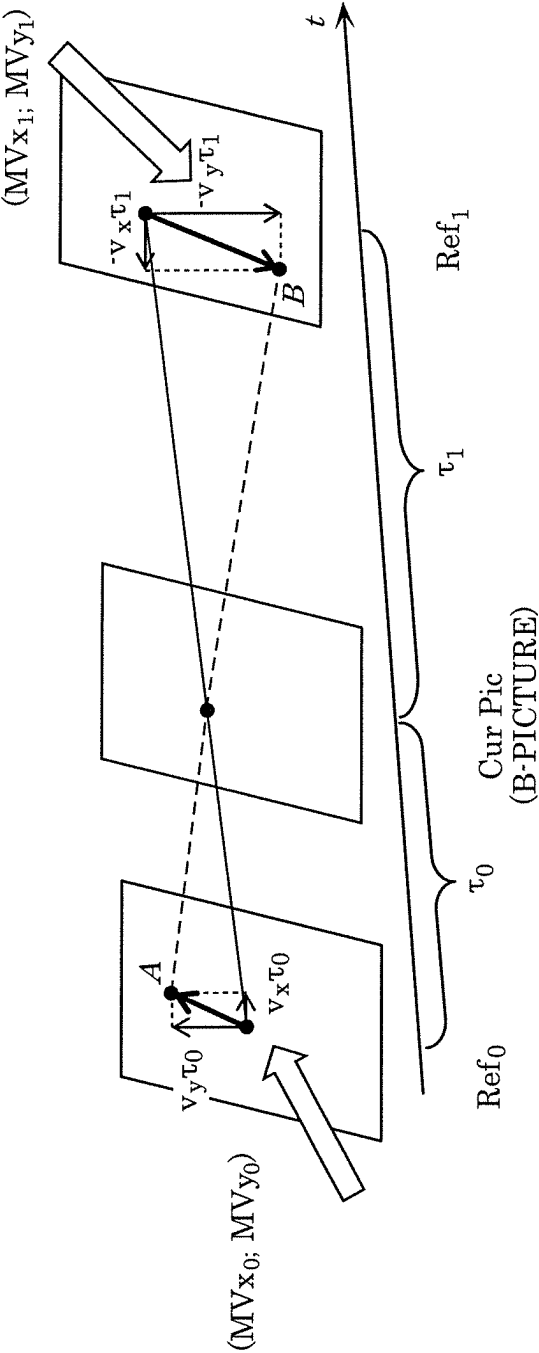


FIG. 9A

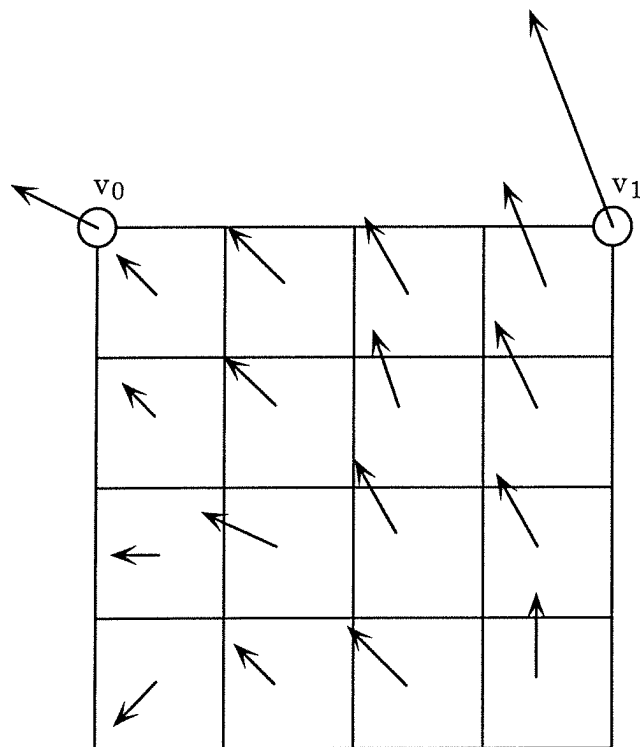


FIG. 9B

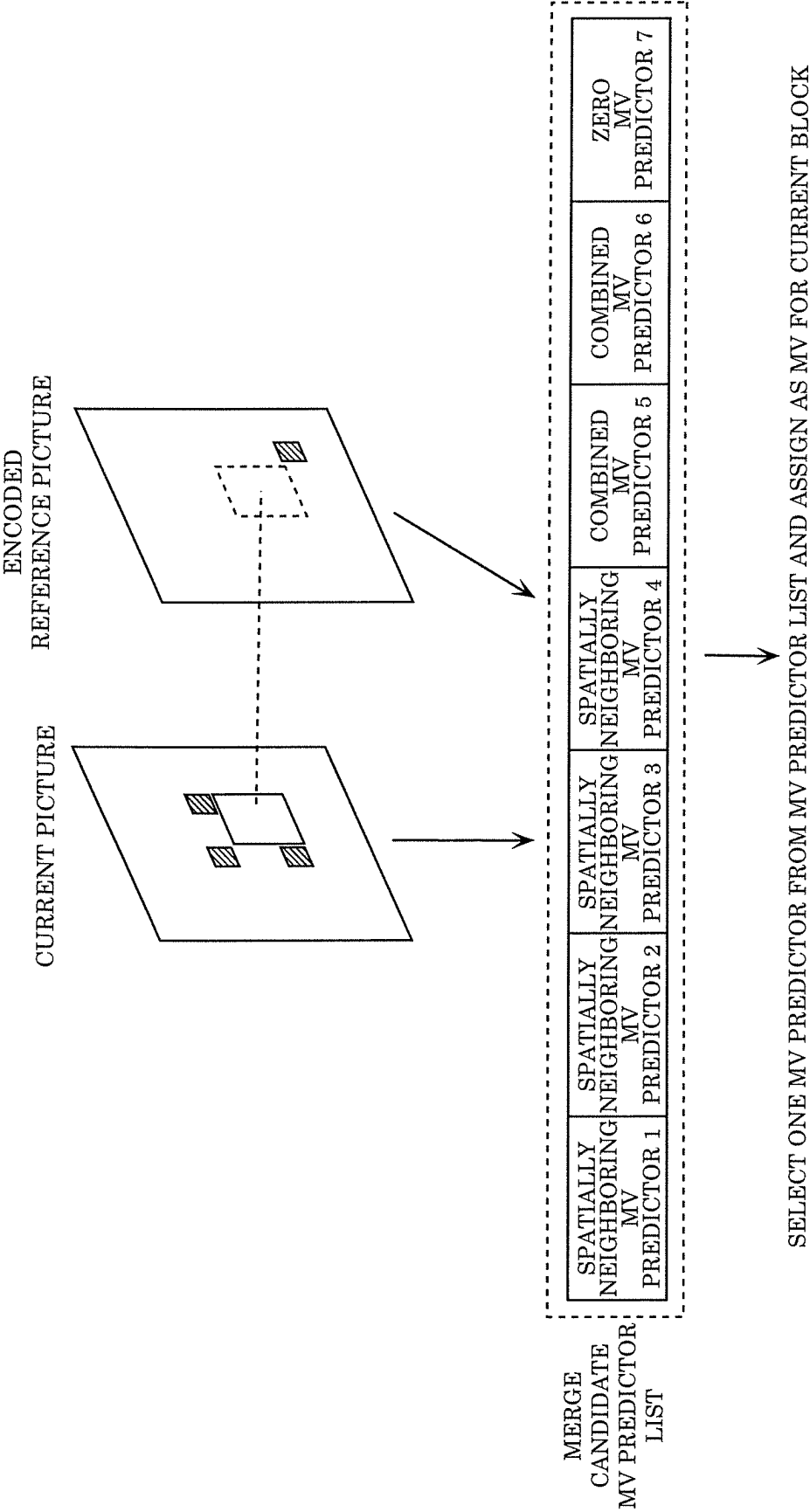


FIG. 9C

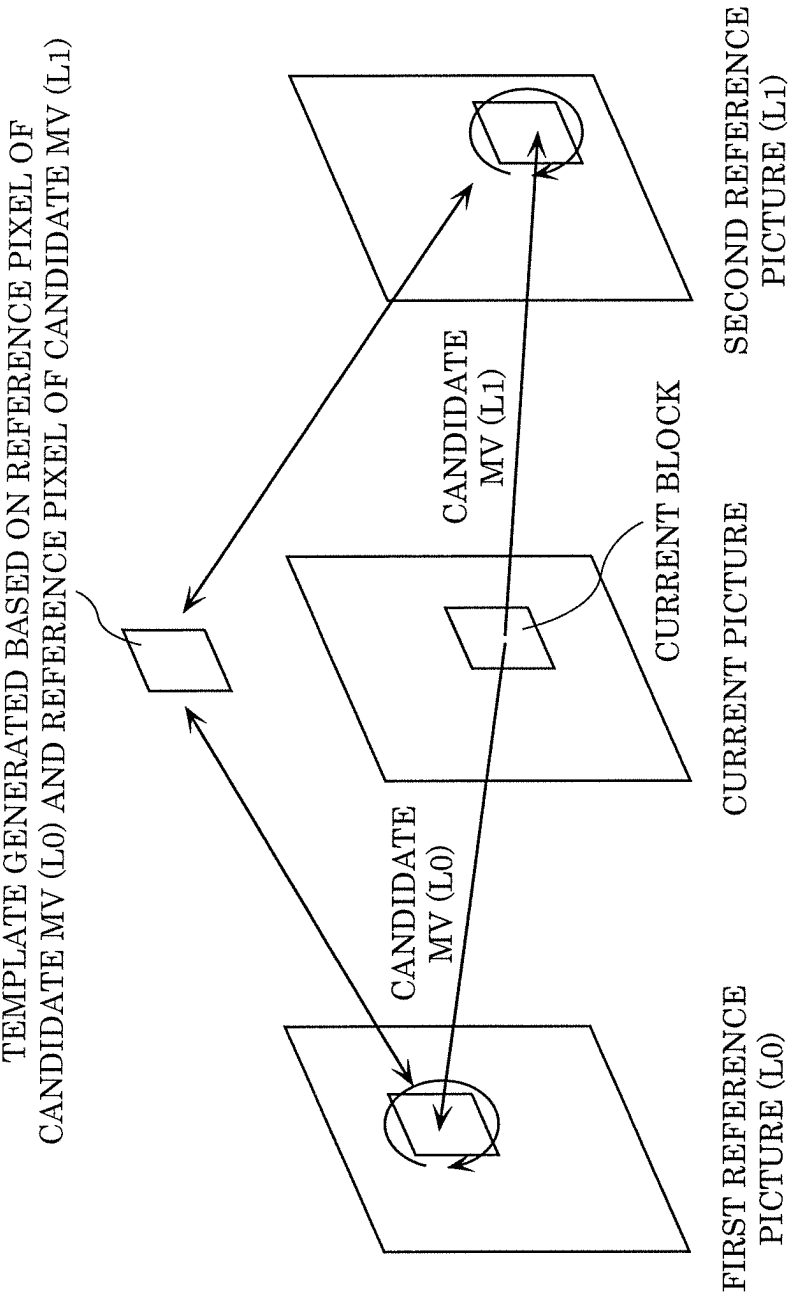


FIG. 9D

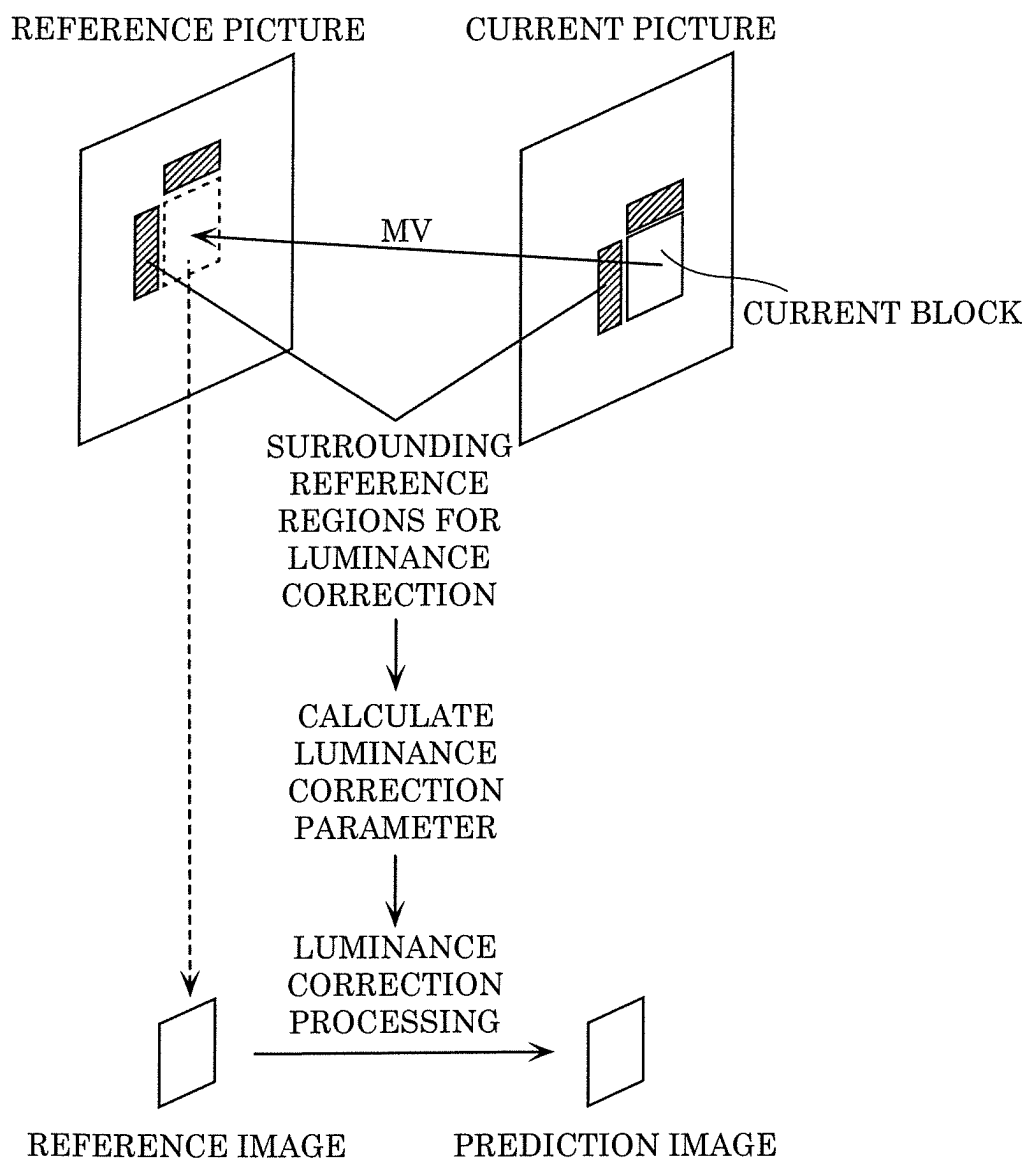


FIG. 10

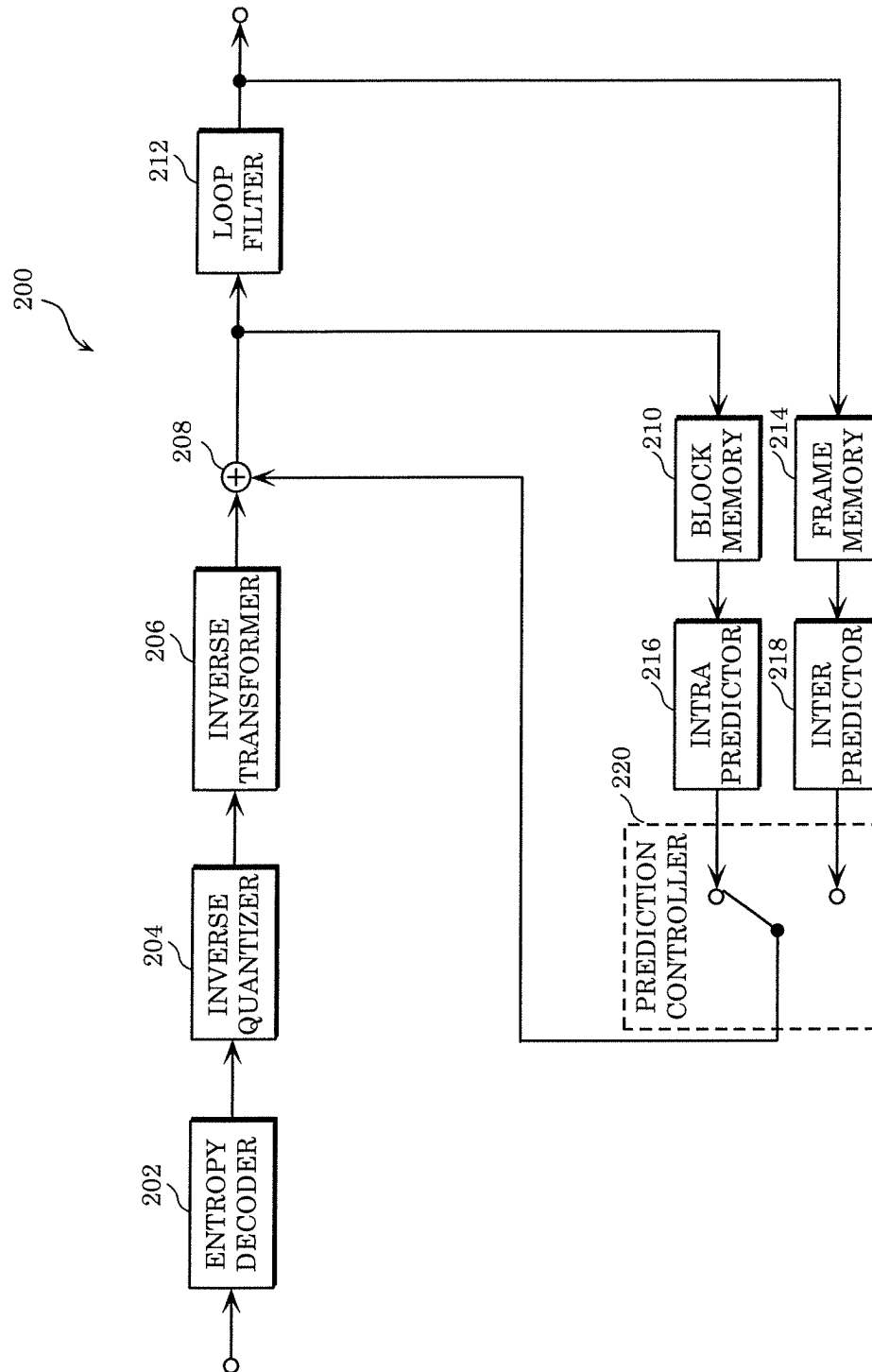


FIG. 11

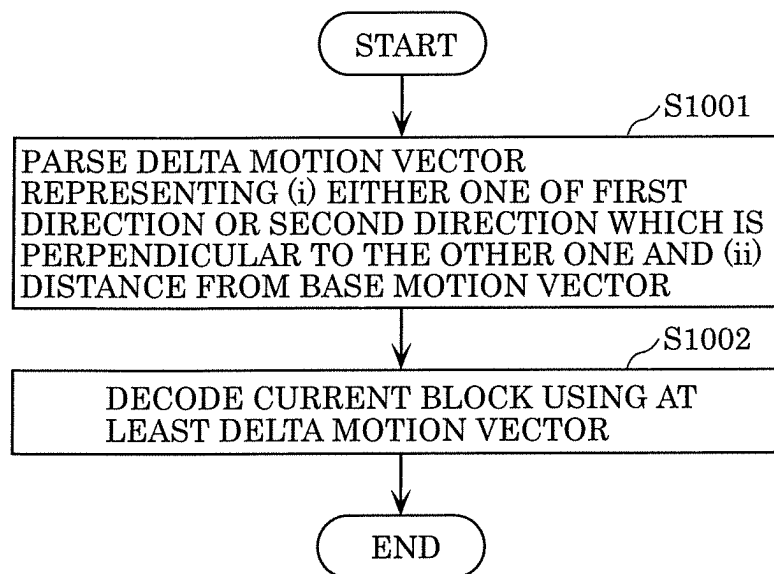


FIG. 12

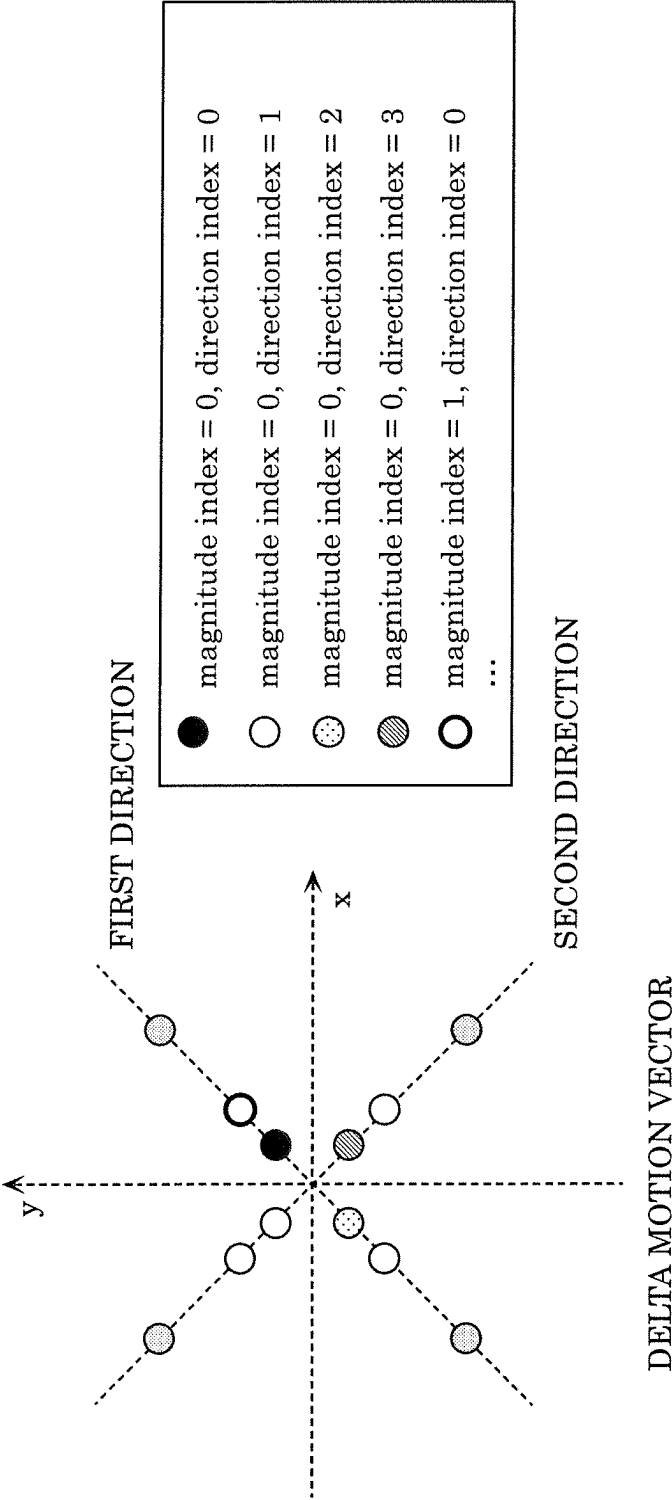


FIG. 13

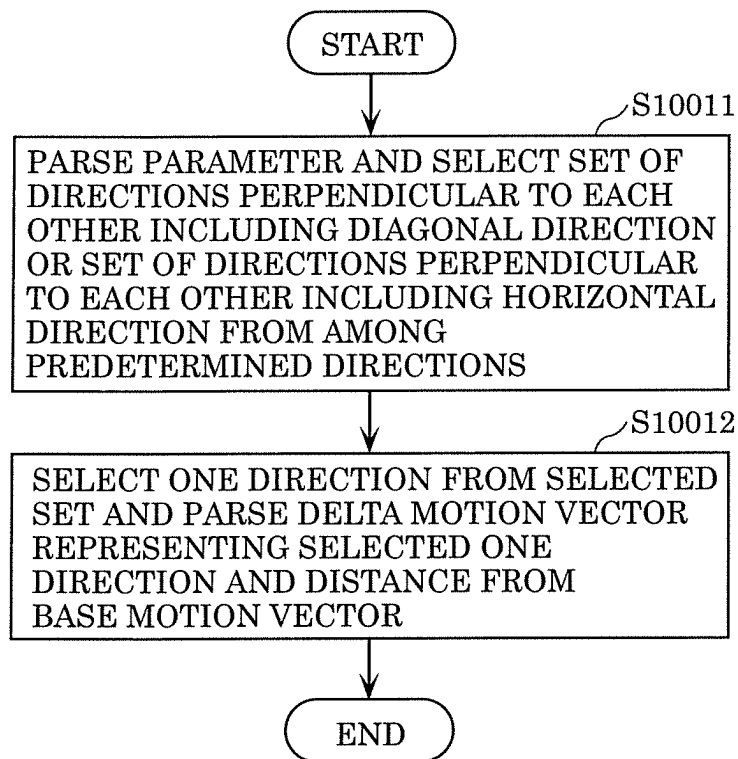


FIG. 14

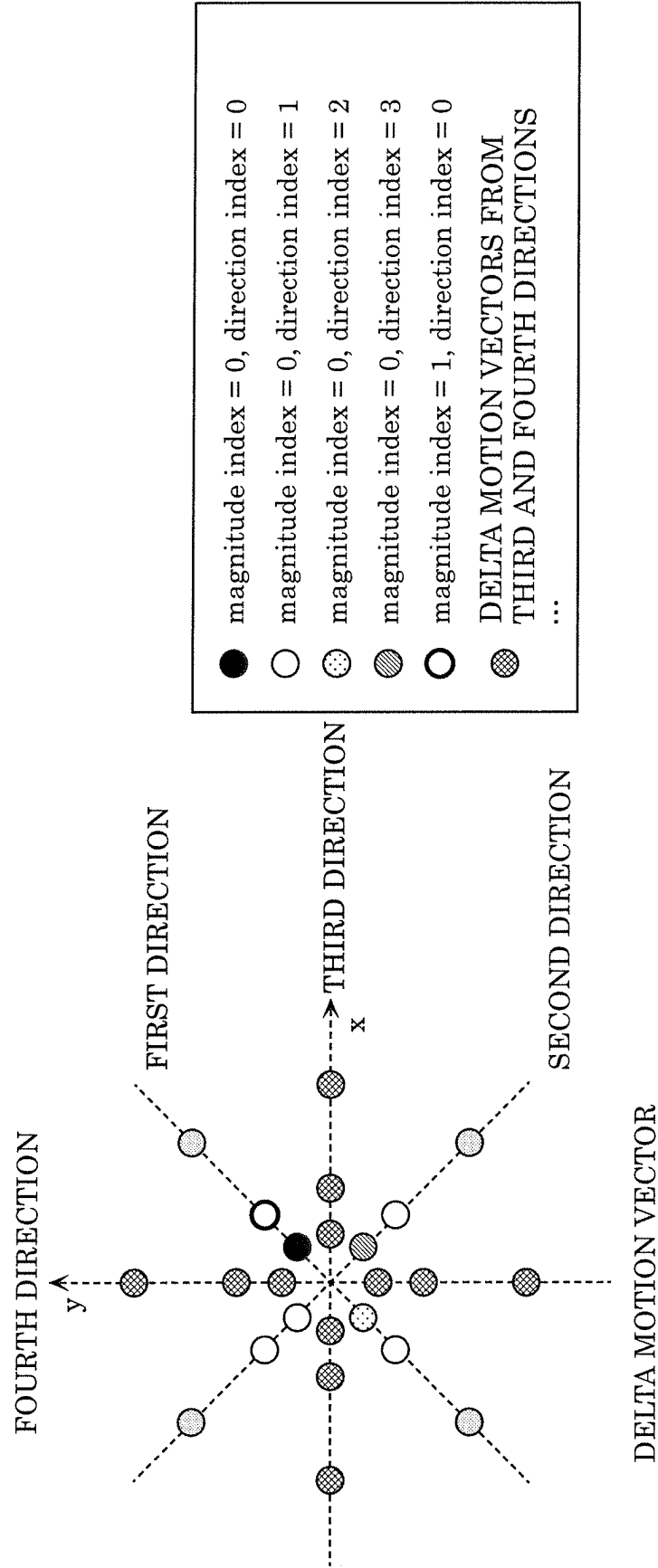


FIG. 15

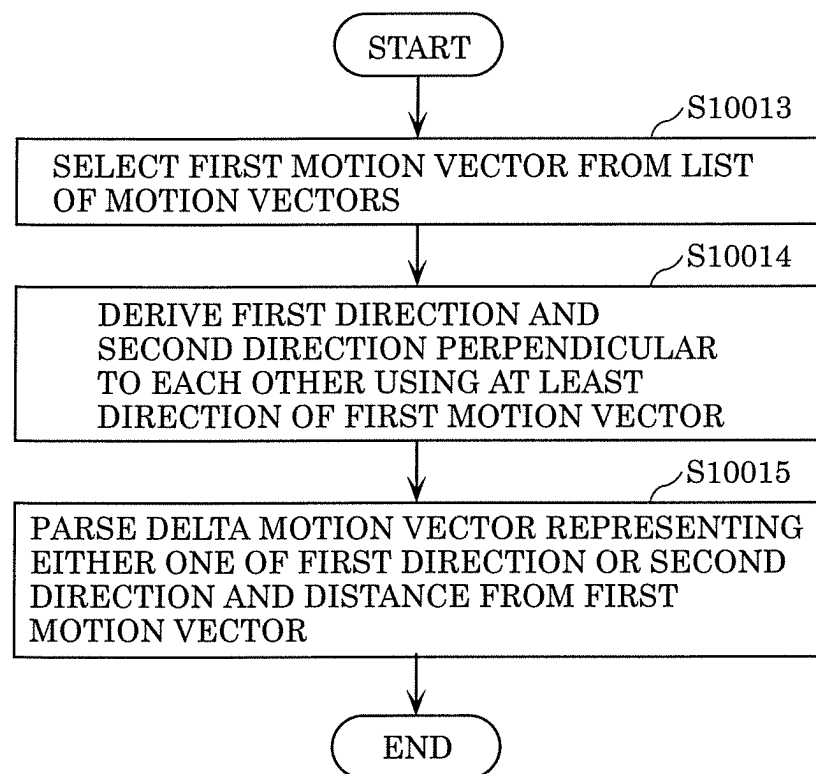


FIG. 16

CANDIDATE IDX	0	1	2	3
N th MVP	1 st MVP	2 nd MVP	3 rd MVP	4 th MVP

FIG. 17

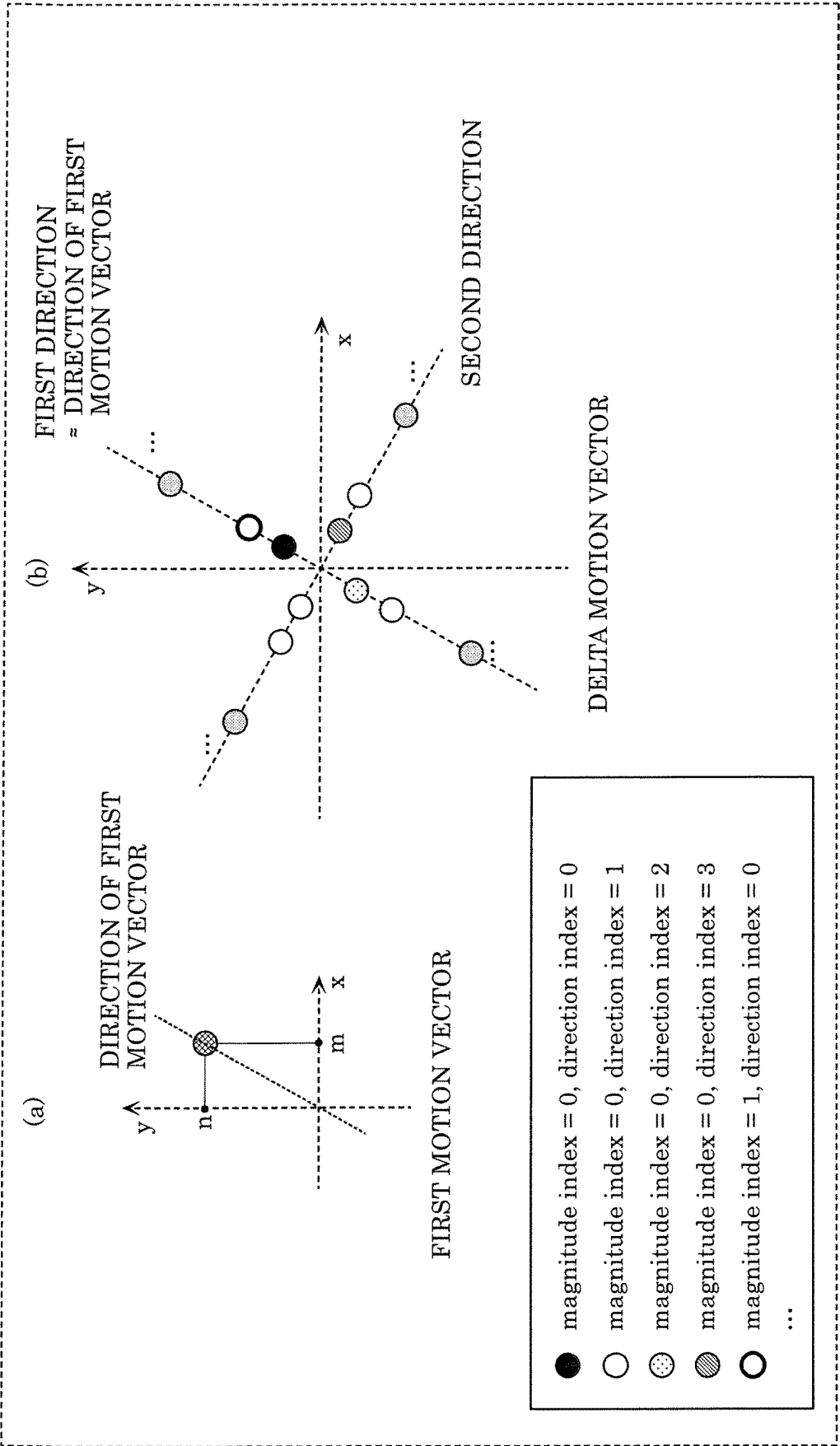


FIG. 18

FIRST VALUE	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
SECOND VALUE	512	512	512	511	511	510	510	509	508	507	506	505	503	502	500	498	497	495	493	491	489	486
THIRD VALUE	0	8	16	24	32	40	48	56	64	71	79	87	94	102	109	117	124	131	139	146	153	160

22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43
484	482	479	477	474	472	469	466	464	461	458	455	452	449	446	443	440	437	434	431	428	425
166	173	180	186	193	199	205	211	217	223	229	235	240	246	251	256	261	266	271	276	281	286

44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64
422	419	416	413	410	407	403	400	397	394	391	388	385	382	379	376	374	371	368	365	362
290	294	299	303	307	311	315	319	323	327	330	334	337	341	344	347	350	353	356	359	362

FIG. 19

MAGNITUDE IDX	0	1	2	3	4	5	6	7
MAGNITUDE	1/4-pel	1/2-pel	1-pel	2-pel	4-pel	8-pel	16-pel	32-pel
SHIFT VALUE	7	6	5	4	3	2	1	0

FIG. 20

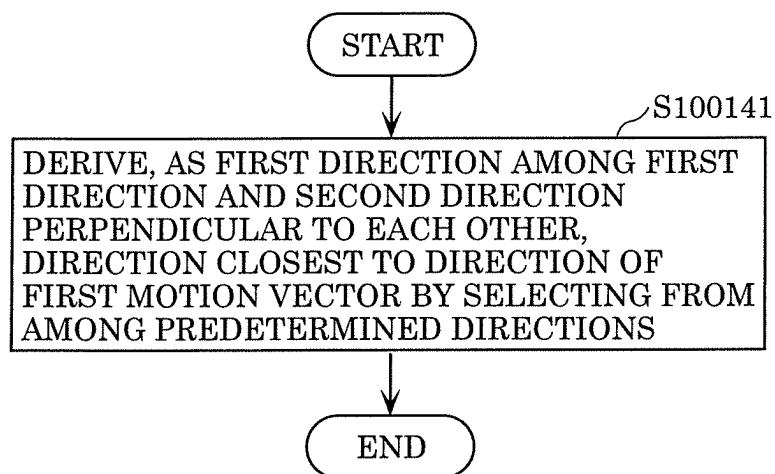


FIG. 21

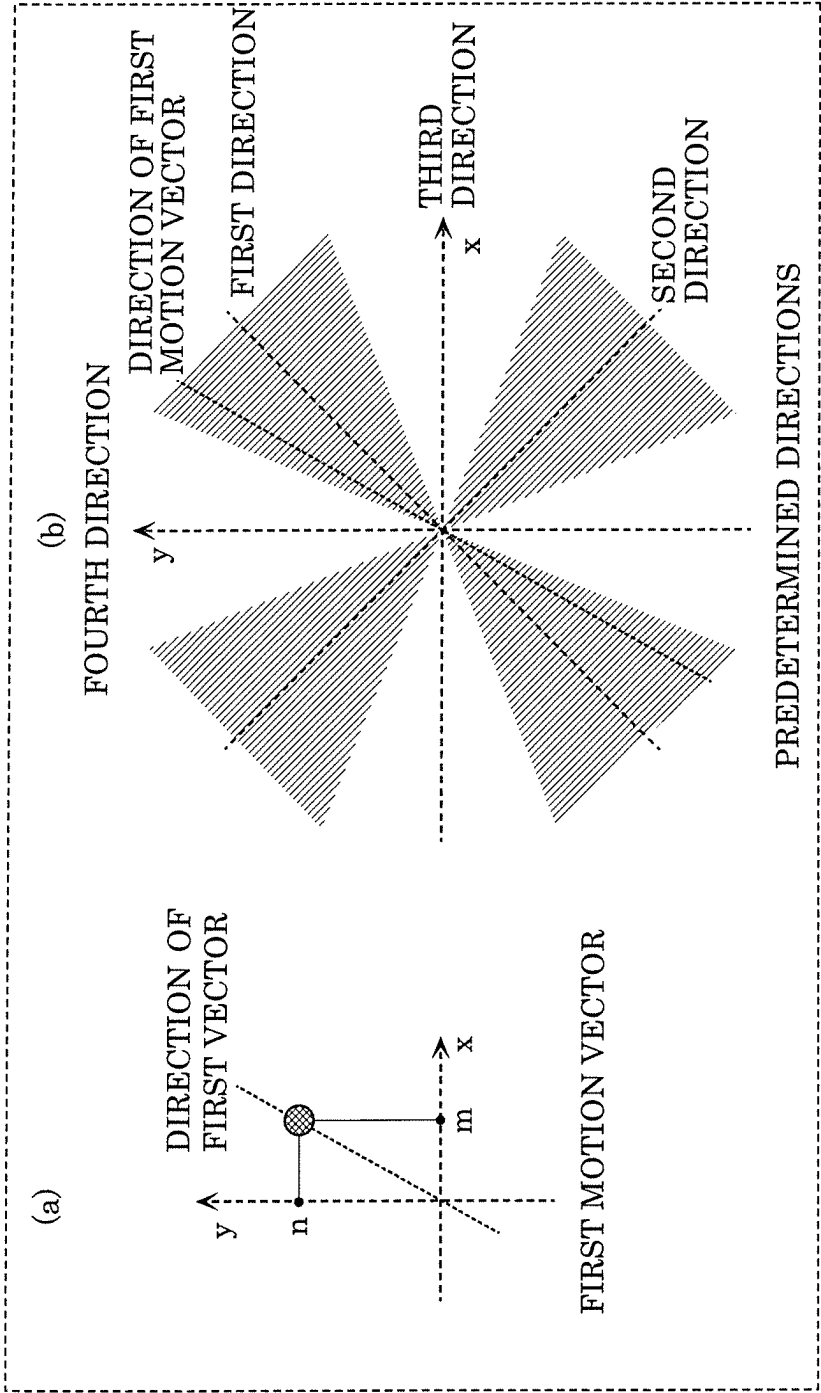


FIG. 22

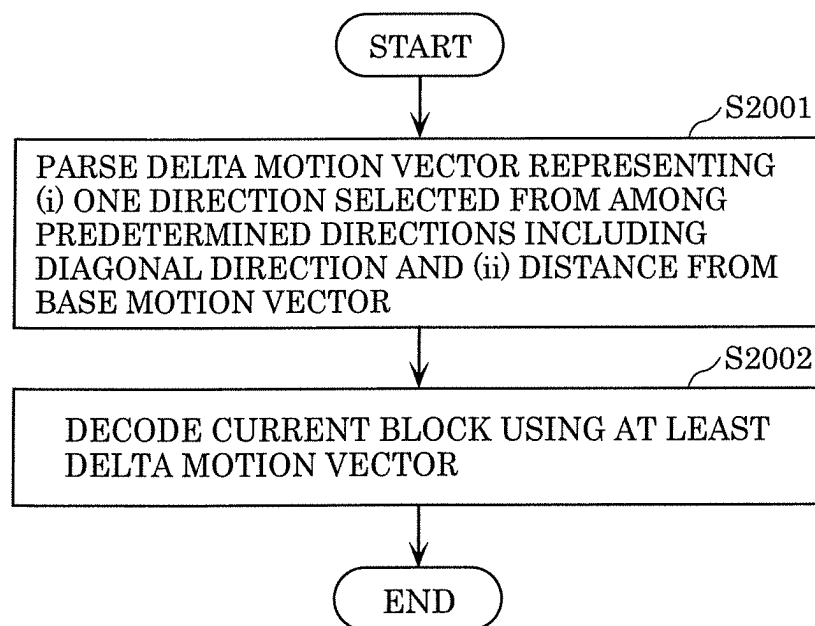


FIG. 23

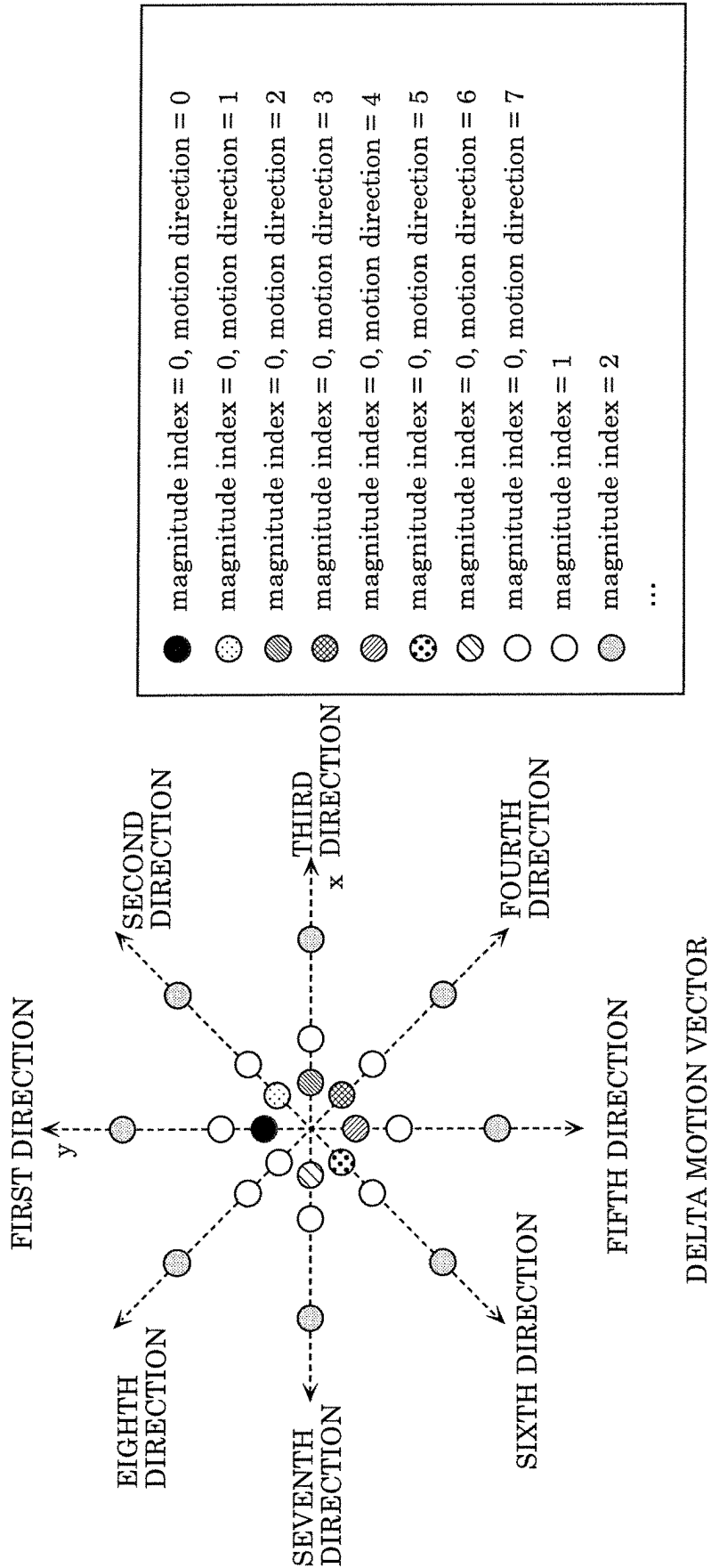


FIG.24

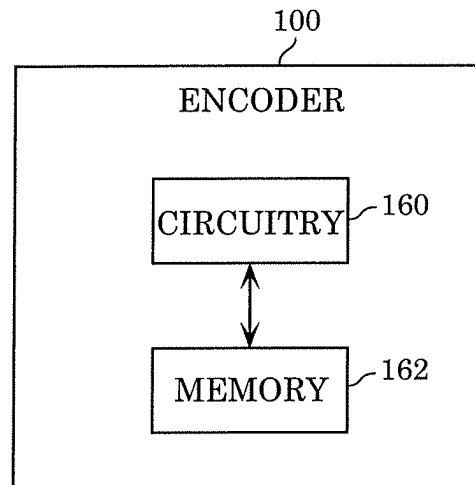


FIG.25

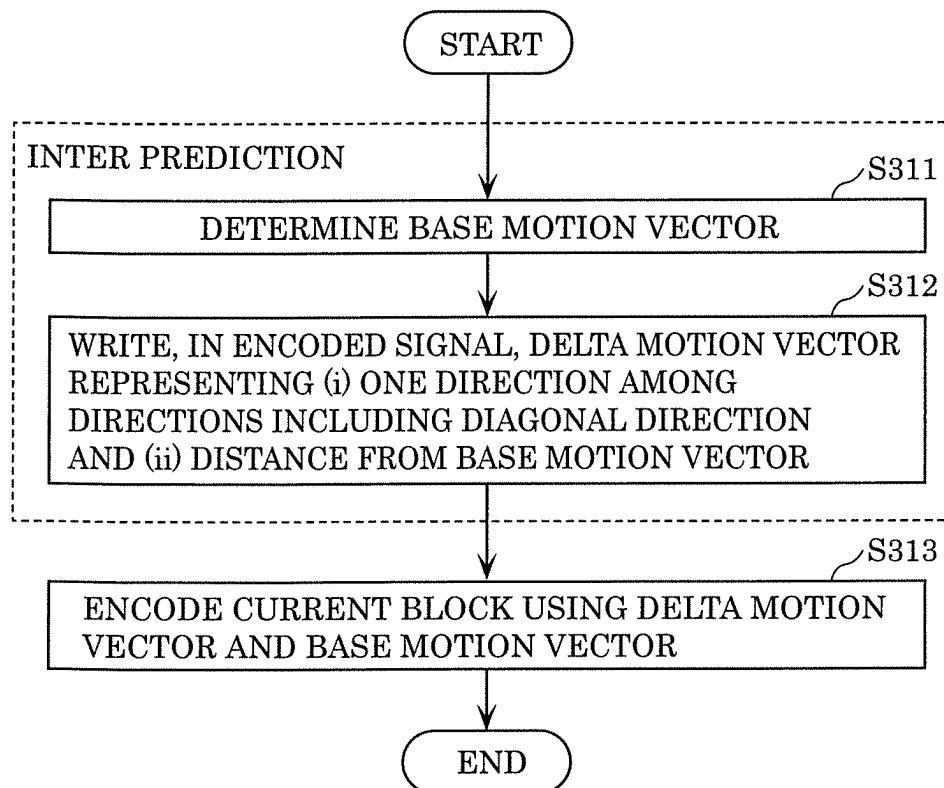


FIG. 26

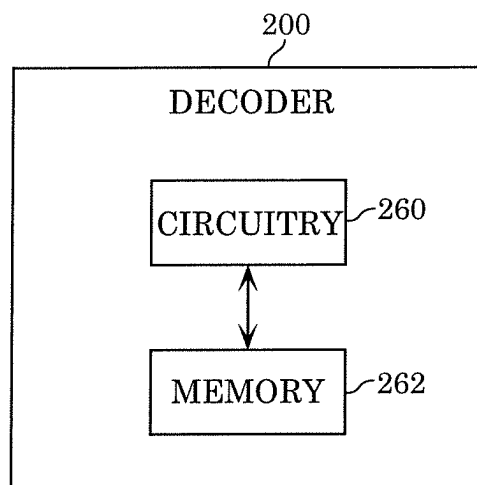


FIG. 27

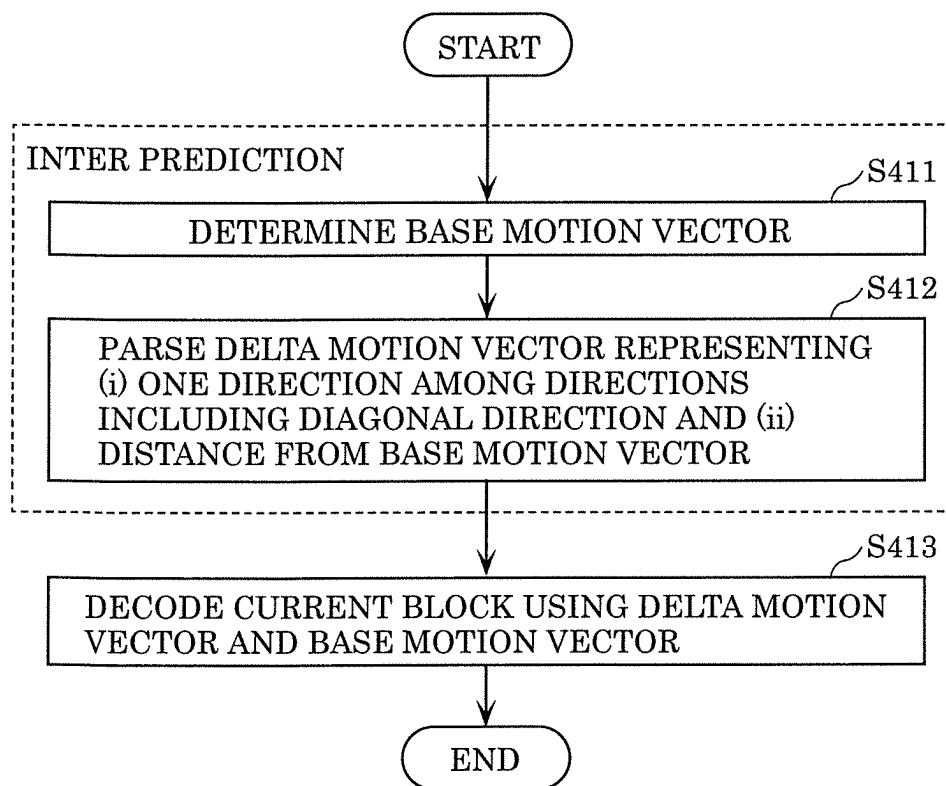


FIG. 28

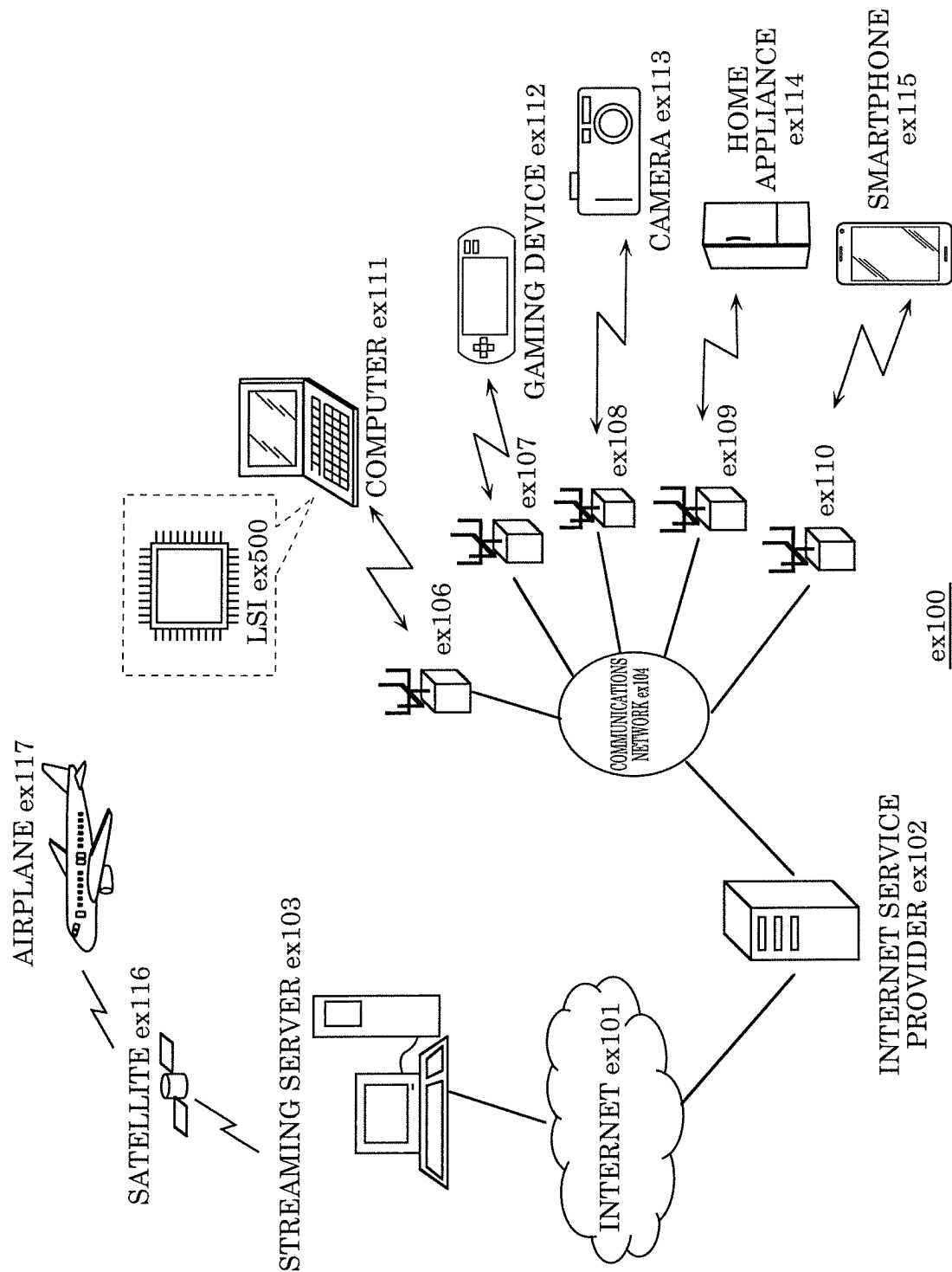


FIG. 29

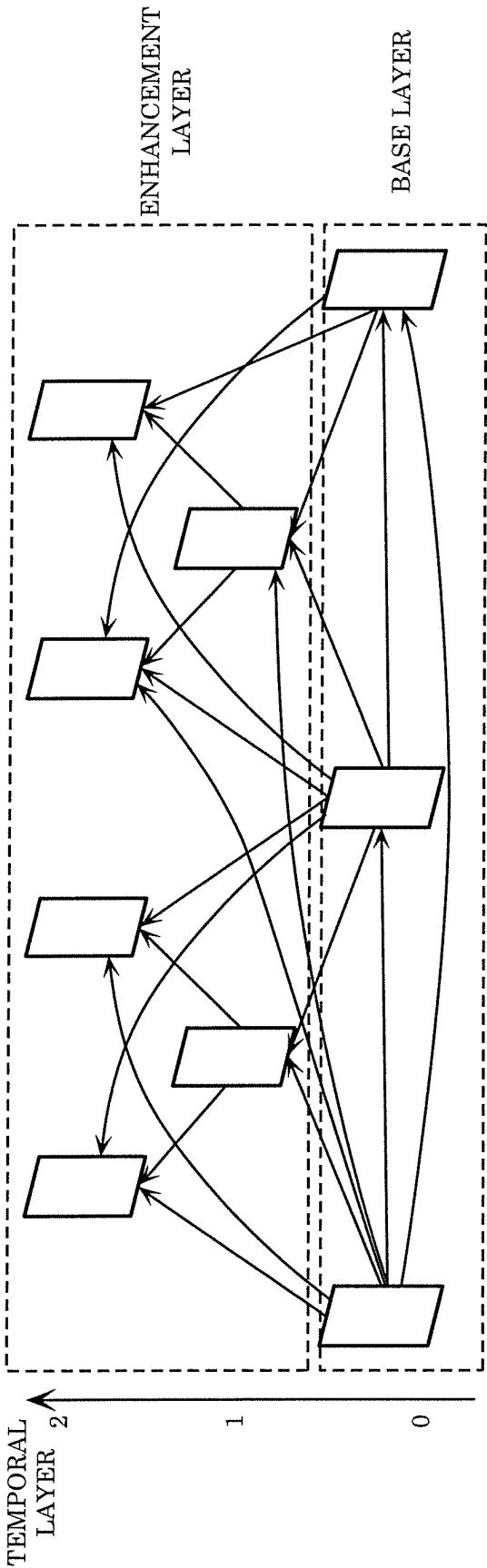


FIG. 30

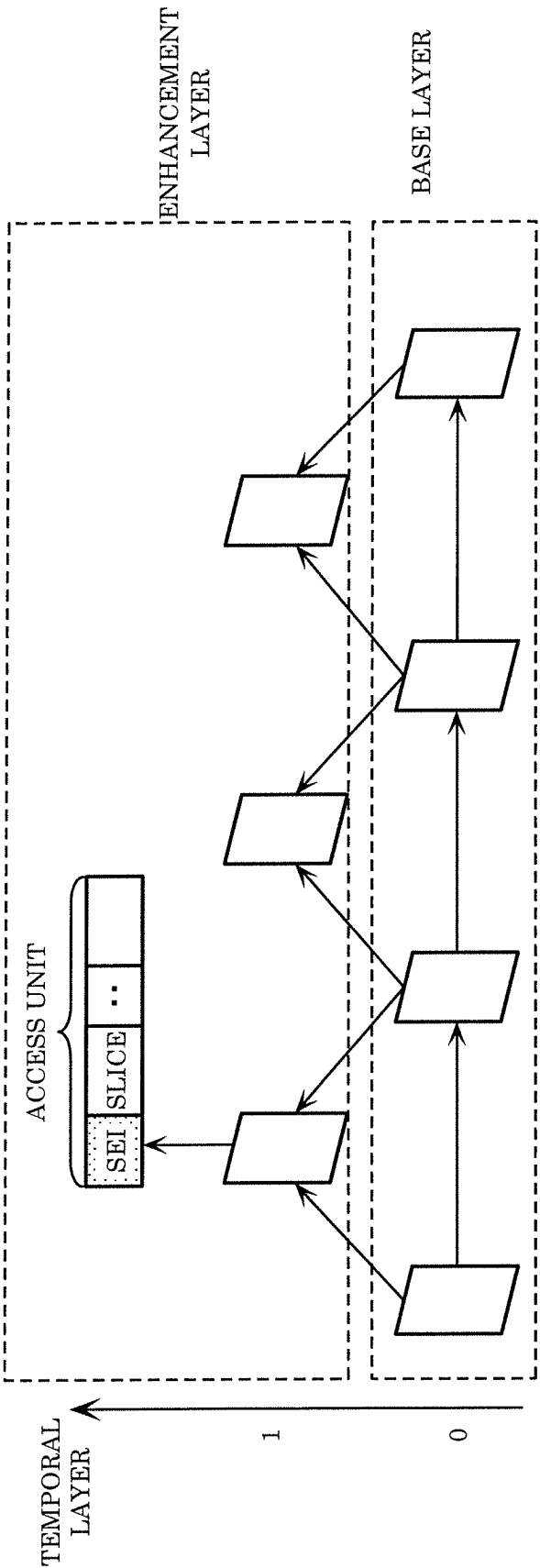


FIG. 31

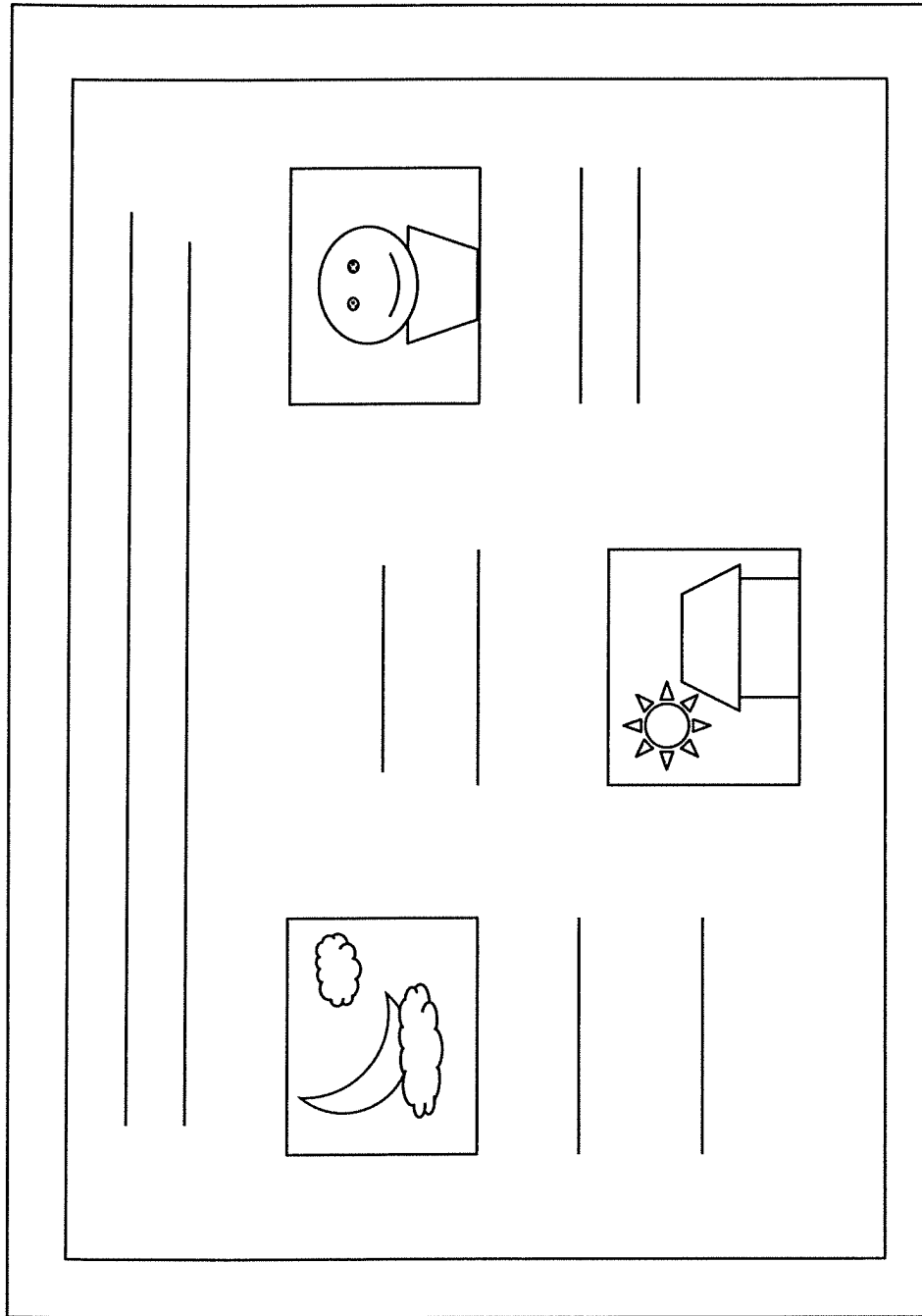


FIG. 32

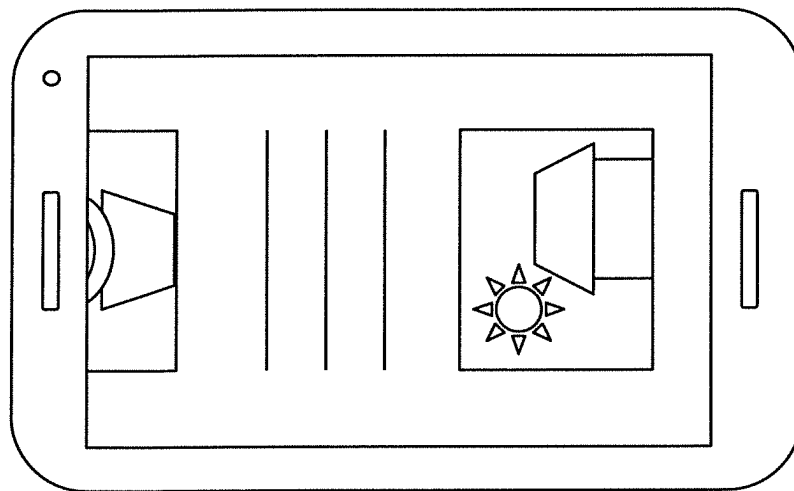


FIG. 33

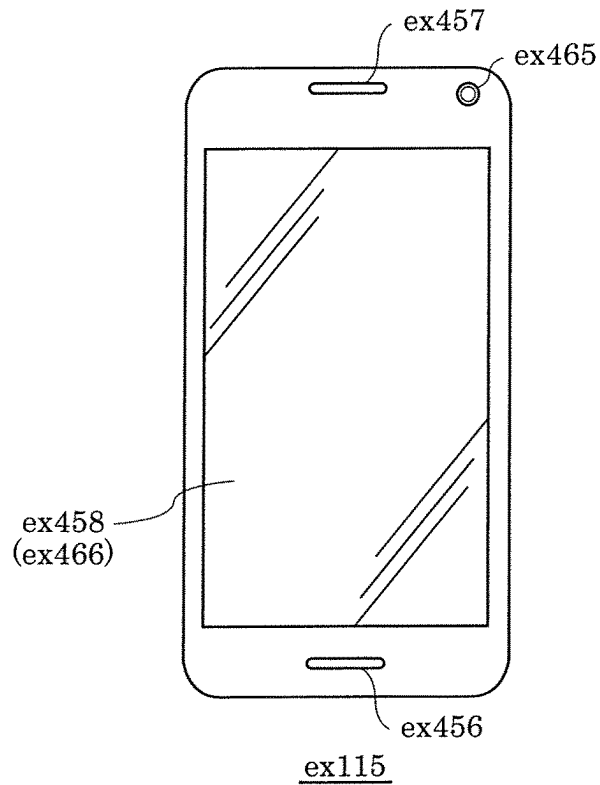
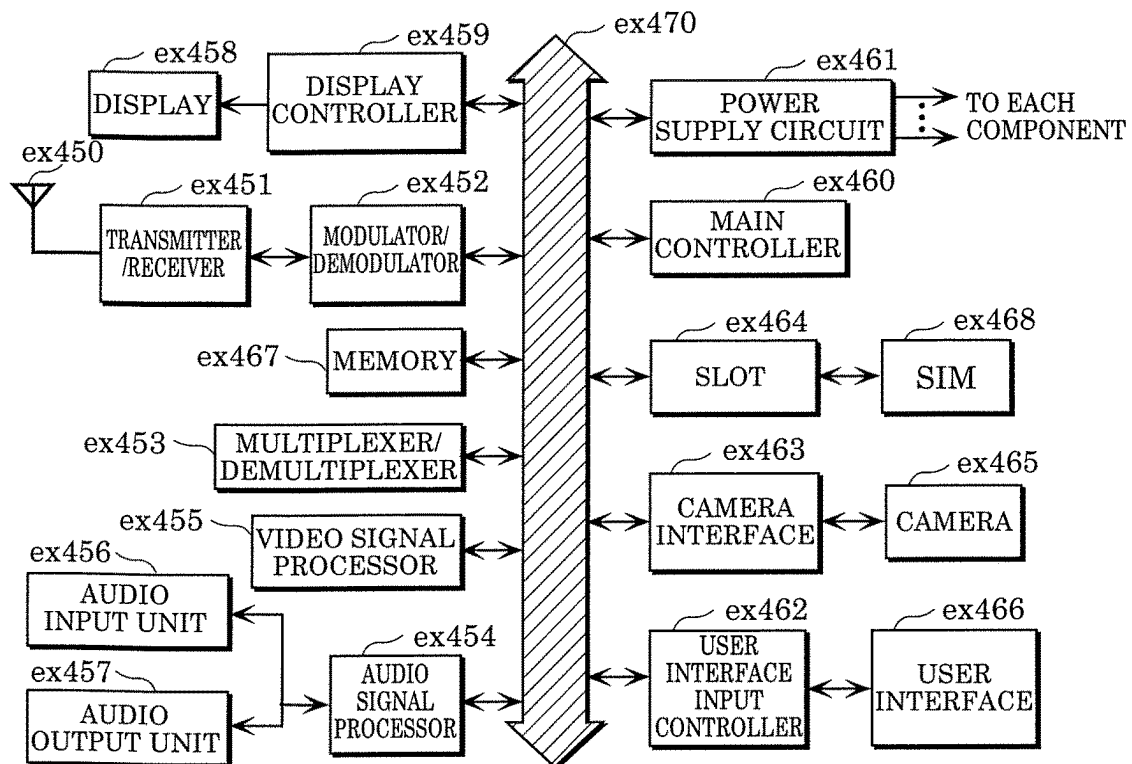


FIG. 34



1

ENCODER, DECODER, ENCODING METHOD, AND DECODING METHOD

CROSS REFERENCE TO RELATED APPLICATIONS

This application is a continuation of U.S. application Ser. No. 17/539,348 filed on Dec. 1, 2021, which is a continuation of U.S. application Ser. No. 17/077,450, now U.S. Pat. No. 11,223,835, filed on Oct. 22, 2020, which is a continuation application of PCT International Patent Application Number PCT/JP2019/016574 filed on Apr. 18, 2019, claiming the benefit of priority of U.S. Provisional Application No. 62/662,524 filed on Apr. 25, 2018. The entire disclosures of the above-identified applications, including the specifications, drawings, and claims are incorporated herein by reference in their entirety.

BACKGROUND

1. Technical Field

The present disclosure relates to an encoder, a decoder, an encoding method, and a decoding method.

2. Description of the Related Art

There has been H.265 as the video coding standard. H.265 is also referred to as High-Efficiency Video Coding (HEVC).

SUMMARY

An encoder according to an aspect of the present disclosure includes circuitry and memory. Using the memory, the circuitry: in inter prediction for a current block, determines a base motion vector, and writes, in an encoded signal, a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and encodes the current block using the delta motion vector and the base motion vector.

A decoder according to an aspect of the present disclosure includes circuitry and memory. Using the memory, the circuitry: in inter prediction for a current block, determines a base motion vector, and parses a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and decodes the current block using the delta motion vector and the base motion vector.

Note that these general and specific aspects may be implemented using a system, a device, a method, an integrated circuit, a computer program, a computer-readable recording medium such as a CD-ROM, or any combination of systems, devices, methods, integrated circuits, computer programs or recording media.

Additional benefits and advantages of the disclosed embodiments will be apparent from the Specification and Drawings. The benefits and/or advantages may be individually obtained by the various embodiments and features of the Specification and Drawings, which need not all be provided in order to obtain one or more of such benefits and/or advantages.

BRIEF DESCRIPTION OF DRAWINGS

These and other objects, advantages and features of the disclosure will become apparent from the following descrip-

2

tion thereof taken in conjunction with the accompanying drawings that illustrate a specific embodiment of the present disclosure.

FIG. 1 is a block diagram illustrating a functional configuration of an encoder according to Embodiment 1;

FIG. 2 illustrates one example of block splitting according to Embodiment 1;

FIG. 3 is a chart indicating transform basis functions for each transform type;

FIG. 4A illustrates one example of a filter shape used in ALF;

FIG. 4B illustrates another example of a filter shape used in ALF;

FIG. 4C illustrates another example of a filter shape used in ALF;

FIG. 5A illustrates 67 intra prediction modes used in intra prediction;

FIG. 5B is a flow chart for illustrating an outline of a prediction image correction process performed via OBMC processing;

FIG. 5C is a conceptual diagram for illustrating an outline of a prediction image correction process performed via OBMC processing;

FIG. 5D illustrates one example of FRUC;

FIG. 6 is for illustrating pattern matching (bilateral matching) between two blocks along a motion trajectory;

FIG. 7 is for illustrating pattern matching (template matching) between a template in the current picture and a block in a reference picture;

FIG. 8 is for illustrating a model assuming uniform linear motion;

FIG. 9A is for illustrating deriving a motion vector of each sub-block based on motion vectors of neighboring blocks;

FIG. 9B is for illustrating an outline of a process for deriving a motion vector via merge mode;

FIG. 9C is a conceptual diagram for illustrating an outline of DMVR processing;

FIG. 9D is for illustrating an outline of a prediction image generation method using a luminance correction process performed via LIC processing;

FIG. 10 is a block diagram illustrating a functional configuration of a decoder according to Embodiment 1;

FIG. 11 is a flow chart illustrating an example of the internal processing of a decoder according to aspect 1 of Embodiment 1;

FIG. 12 is for illustrating a delta motion vector used in inter prediction according to aspect 1 of Embodiment 1;

FIG. 13 is a flow chart illustrating an example of the internal processing of a decoder according to aspect 2 of Embodiment 1;

FIG. 14 is for illustrating a delta motion vector used in inter prediction according to aspect 2 of Embodiment 1;

FIG. 15 is a flow chart illustrating an example of the internal processing of a decoder according to aspect 3 of Embodiment 1;

FIG. 16 is a diagram illustrating an example of a MV predictor list according to aspect 3 of Embodiment 1;

FIG. 17 is for illustrating a delta motion vector used in inter prediction according to aspect 3 of Embodiment 1;

FIG. 18 is a diagram illustrating an example of a look up table according to aspect 3 of Embodiment 1;

FIG. 19 is a table illustrating an example of a relationship between magnitude indexes and shift values in accordance with pixel precision according to aspect 3 of Embodiment 1;

FIG. 20 is a flow chart illustrating an example of the internal processing of a decoder according to aspect 4 of Embodiment 1;

3

FIG. 21 is for illustrating a delta motion vector used in inter prediction according to aspect 4 of Embodiment 1;

FIG. 22 is a flow chart illustrating an example of the internal processing of a decoder according to aspect 5 of Embodiment 1;

FIG. 23 is for illustrating a delta motion vector used in inter prediction according to aspect 5 of Embodiment 1;

FIG. 24 is a block diagram illustrating an implementation example of the encoder according to Embodiment 1;

FIG. 25 is a flow chart illustrating an operation example of the encoder according to Embodiment 1;

FIG. 26 is a block diagram illustrating an implementation example of the decoder according to Embodiment 1;

FIG. 27 is a flow chart illustrating an operation example of the decoder according to Embodiment 1;

FIG. 28 illustrates an overall configuration of a content providing system for implementing a content distribution service;

FIG. 29 illustrates one example of an encoding structure in scalable encoding;

FIG. 30 illustrates one example of an encoding structure in scalable encoding;

FIG. 31 illustrates an example of a display screen of a web page;

FIG. 32 illustrates an example of a display screen of a web page;

FIG. 33 illustrates one example of a smartphone; and

FIG. 34 is a block diagram illustrating a configuration example of a smartphone.

DETAILED DESCRIPTION OF THE EMBODIMENTS

For example, an encoder according to an aspect of the present disclosure includes circuitry and memory. Using the memory, the circuitry: in inter prediction for a current block, determines a base motion vector, and writes, in an encoded signal, a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and encodes the current block using the delta motion vector and the base motion vector.

In this way, the encoder derives, in inter prediction, a higher-accuracy motion vector. With this, it is possible to improve encoding efficiency of inter prediction. Accordingly, the encoder can improve encoding efficiency.

Here, for example, the inter prediction for the current block is inter prediction in merge mode, and the circuitry determines the base motion vector by selecting one candidate motion vector from a list indicating a plurality of candidate motion vectors for the current block.

Moreover, for example, the plurality of directions are predetermined, in the inter prediction, the circuitry: selects a set of a first direction and a second direction from among the plurality of directions including the diagonal direction, based on an obtained prediction parameter, the first direction and the second direction being perpendicular to each other; and selects, as the one direction, either one of the first direction or the second direction of the set selected.

Moreover, for example, in the inter prediction, the circuitry: selects, as the base motion vector, a first motion vector from a list indicating a plurality of candidate motion vectors for the current block; derives, as the plurality of directions, a first direction and a second direction using a direction of the first motion vector, the first direction and the second direction being perpendicular to each other; and

4

parses the delta motion vector using either one of the first direction or the second direction as the one direction.

Moreover, for example, in the inter prediction, the circuitry derives the first direction using the direction of the first motion vector to derive the first direction and the second direction perpendicular to each other, the first direction being substantially the same as the direction of the first motion vector.

Moreover, for example, the plurality of directions are predetermined, in the inter prediction, the circuitry derives, as the first direction, a direction closest to the direction of the first motion vector among the plurality of directions.

Moreover, for example, a decoder according to an aspect of the present disclosure includes circuitry and memory. Using the memory, the circuitry: in inter prediction for a current block, determines a base motion vector, and parses a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and decodes the current block using the delta motion vector and the base motion vector.

Moreover, the inter prediction for the current block is inter prediction in merge mode, and the circuitry determines the base motion vector by selecting one candidate motion vector from a list indicating a plurality of candidate motion vectors for the current block.

In this way, the decoder derives, in inter prediction, a higher-accuracy motion vector. With this, it is possible to improve encoding efficiency of inter prediction. Accordingly, the decoder can improve encoding efficiency.

Moreover, for example, the plurality of directions are predetermined, in the inter prediction, the circuitry: selects a set of a first direction and a second direction from among the plurality of directions including the diagonal direction, based on an obtained prediction parameter, the first direction and the second direction being perpendicular to each other; and selects, as the one direction, either one of the first direction or the second direction of the set selected.

Moreover, for example, in the inter prediction, the circuitry: selects, as the base motion vector, a first motion vector from a list indicating a plurality of candidate motion vectors for the current block; derives, as the plurality of directions, a first direction and a second direction using a direction of the first motion vector, the first direction and the second direction being perpendicular to each other; and parses the delta motion vector using either one of the first direction or the second direction as the one direction.

Moreover, for example, in the inter prediction, the circuitry derives the first direction using the direction of the first motion vector to derive the first direction and the second direction perpendicular to each other, the first direction being substantially the same as the direction of the first motion vector.

Moreover, for example, the plurality of directions are predetermined, in the inter prediction, the circuitry derives, as the first direction, a direction closest to the direction of the first motion vector among the plurality of directions.

Moreover, for example, the plurality of directions are predetermined, in the inter prediction, the circuitry selects the one direction among the plurality of directions using a flag included in the obtained prediction parameter.

Moreover, for example, an encoding method according to an aspect of the present disclosure includes in inter prediction for a current block, determining a base motion vector, and writing, in an encoded signal, a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the

5

base motion vector; and encoding the current block using the delta motion vector and the base motion vector.

In this way, the encoding method derives, in inter prediction, a higher-accuracy motion vector. With this, it is possible to improve encoding efficiency of inter prediction. Accordingly, the encoding method can improve encoding efficiency.

Moreover, for example, a decoding method according to an aspect of the present disclosure includes in inter prediction for a current block, determining a base motion vector, and parsing a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and decoding the current block using the delta motion vector and the base motion vector.

In this way, the decoding method derives, in inter prediction, a higher-accuracy motion vector. With this, it is possible to improve encoding efficiency of inter prediction. Accordingly, the decoding method can improve encoding efficiency.

Furthermore, these general and specific aspects may be implemented using a system, a device, a method, an integrated circuit, a computer program, a computer-readable recording medium such as a CD-ROM, or any combination of systems, devices, methods, integrated circuits, computer programs or recording media.

Hereinafter, embodiments will be described with reference to the drawings.

Note that the embodiments described below each show a general or specific example. The numerical values, shapes, materials, components, the arrangement and connection of the components, steps, order of the steps, etc., indicated in the following embodiments are mere examples, and therefore are not intended to limit the scope of the claims. Therefore, among the components in the following embodiments, those not recited in any of the independent claims defining the broadest inventive concepts are described as optional components.

Embodiment 1

First, an outline of Embodiment 1 will be presented. Embodiment 1 is one example of an encoder and a decoder to which the processes and/or configurations presented in subsequent description of aspects of the present disclosure are applicable. Note that Embodiment 1 is merely one example of an encoder and a decoder to which the processes and/or configurations presented in the description of aspects of the present disclosure are applicable. The processes and/or configurations presented in the description of aspects of the present disclosure can also be implemented in an encoder and a decoder different from those according to Embodiment 1.

When the processes and/or configurations presented in the description of aspects of the present disclosure are applied to Embodiment 1, for example, any of the following may be performed.

- (1) regarding the encoder or the decoder according to Embodiment 1, among components included in the encoder or the decoder according to Embodiment 1, substituting a component corresponding to a component presented in the description of aspects of the present disclosure with a component presented in the description of aspects of the present disclosure;
- (2) regarding the encoder or the decoder according to Embodiment 1, implementing discretionary changes to functions or implemented processes performed by one

6

or more components included in the encoder or the decoder according to Embodiment 1, such as addition, substitution, or removal, etc., of such functions or implemented processes, then substituting a component corresponding to a component presented in the description of aspects of the present disclosure with a component presented in the description of aspects of the present disclosure;

- (3) regarding the method implemented by the encoder or the decoder according to Embodiment 1, implementing discretionary changes such as addition of processes and/or substitution, removal of one or more of the processes included in the method, and then substituting a processes corresponding to a process presented in the description of aspects of the present disclosure with a process presented in the description of aspects of the present disclosure;
- (4) combining one or more components included in the encoder or the decoder according to Embodiment 1 with a component presented in the description of aspects of the present disclosure, a component including one or more functions included in a component presented in the description of aspects of the present disclosure, or a component that implements one or more processes implemented by a component presented in the description of aspects of the present disclosure;
- (5) combining a component including one or more functions included in one or more components included in the encoder or the decoder according to Embodiment 1, or a component that implements one or more processes implemented by one or more components included in the encoder or the decoder according to Embodiment 1 with a component presented in the description of aspects of the present disclosure, a component including one or more functions included in a component presented in the description of aspects of the present disclosure, or a component that implements one or more processes implemented by a component presented in the description of aspects of the present disclosure;
- (6) regarding the method implemented by the encoder or the decoder according to Embodiment 1, among processes included in the method, substituting a process corresponding to a process presented in the description of aspects of the present disclosure with a process presented in the description of aspects of the present disclosure; and
- (7) combining one or more processes included in the method implemented by the encoder or the decoder according to Embodiment 1 with a process presented in the description of aspects of the present disclosure.

Note that the implementation of the processes and/or configurations presented in the description of aspects of the present disclosure is not limited to the above examples. For example, the processes and/or configurations presented in the description of aspects of the present disclosure may be implemented in a device used for a purpose different from the moving picture/picture encoder or the moving picture/picture decoder disclosed in Embodiment 1. Moreover, the processes and/or configurations presented in the description of aspects of the present disclosure may be independently implemented. Moreover, processes and/or configurations described in different aspects may be combined.

Encoder Outline

First, the encoder according to Embodiment 1 will be outlined. FIG. 1 is a block diagram illustrating a functional

configuration of encoder **100** according to Embodiment 1. Encoder **100** is a moving picture/picture encoder that encodes a moving picture/picture block by block.

As illustrated in FIG. 1, encoder **100** is a device that encodes a picture block by block, and includes splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, block memory **118**, loop filter **120**, frame memory **122**, intra predictor **124**, inter predictor **126**, and prediction controller **128**.

Encoder **100** is realized as, for example, a generic processor and memory. In this case, when a software program stored in the memory is executed by the processor, the processor functions as splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, loop filter **120**, intra predictor **124**, inter predictor **126**, and prediction controller **128**. Alternatively, encoder **100** may be realized as one or more dedicated electronic circuits corresponding to splitter **102**, subtractor **104**, transformer **106**, quantizer **108**, entropy encoder **110**, inverse quantizer **112**, inverse transformer **114**, adder **116**, loop filter **120**, intra predictor **124**, inter predictor **126**, and prediction controller **128**.

Hereinafter, each component included in encoder **100** will be described.

Splitter

Splitter **102** splits each picture included in an input moving picture into blocks, and outputs each block to subtractor **104**. For example, splitter **102** first splits a picture into blocks of a fixed size (for example, 128×128). The fixed size block is also referred to as coding tree unit (CTU). Splitter **102** then splits each fixed size block into blocks of variable sizes (for example, 64×64 or smaller), based on recursive quadtree and/or binary tree block splitting. The variable size block is also referred to as a coding unit (CU), a prediction unit (PU), or a transform unit (TU). Note that in this embodiment, there is no need to differentiate between CU, PU, and TU; all or some of the blocks in a picture may be processed per CU, PU, or TU.

FIG. 2 illustrates one example of block splitting according to Embodiment 1. In FIG. 2, the solid lines represent block boundaries of blocks split by quadtree block splitting, and the dashed lines represent block boundaries of blocks split by binary tree block splitting.

Here, block **10** is a square 128×128 pixel block (128×128 block). This 128×128 block **10** is first split into four square 64×64 blocks (quadtree block splitting).

The top left 64×64 block is further vertically split into two rectangle 32×64 blocks, and the left 32×64 block is further vertically split into two rectangle 16×64 blocks (binary tree block splitting). As a result, the top left 64×64 block is split into two 16×64 blocks **11** and **12** and one 32×64 block **13**.

The top right 64×64 block is horizontally split into two rectangle 64×32 blocks **14** and **15** (binary tree block splitting).

The bottom left 64×64 block is first split into four square 32×32 blocks (quadtree block splitting). The top left block and the bottom right block among the four 32×32 blocks are further split. The top left 32×32 block is vertically split into two rectangle 16×32 blocks, and the right 16×32 block is further horizontally split into two 16×16 blocks (binary tree block splitting). The bottom right 32×32 block is horizontally split into two 32×16 blocks (binary tree block splitting). As a result, the bottom left 64×64 block is split into 16×32

block **16**, two 16×16 blocks **17** and **18**, two 32×32 blocks **19** and **20**, and two 32×16 blocks **21** and **22**.

The bottom right 64×64 block **23** is not split.

As described above, in FIG. 2, block **10** is split into 13 variable size blocks **11** through **23** based on recursive quadtree and binary tree block splitting. This type of splitting is also referred to as quadtree plus binary tree (QTBT) splitting.

Note that in FIG. 2, one block is split into four or two blocks (quadtree or binary tree block splitting), but splitting is not limited to this example. For example, one block may be split into three blocks (ternary block splitting). Splitting including such ternary block splitting is also referred to as multi-type tree (MBT) splitting.

Subtractor

Subtractor **104** subtracts a prediction signal (prediction sample) from an original signal (original sample) per block split by splitter **102**. In other words, subtractor **104** calculates prediction errors (also referred to as residuals) of a block to be encoded (hereinafter referred to as a current block). Subtractor **104** then outputs the calculated prediction errors to transformer **106**.

The original signal is a signal input into encoder **100**, and is a signal representing an image for each picture included in a moving picture (for example, a luma signal and two chroma signals). Hereinafter, a signal representing an image is also referred to as a sample.

Transformer

Transformer **106** transforms spatial domain prediction errors into frequency domain transform coefficients, and outputs the transform coefficients to quantizer **108**. More specifically, transformer **106** applies, for example, a pre-defined discrete cosine transform (DCT) or discrete sine transform (DST) to spatial domain prediction errors.

Note that transformer **106** may adaptively select a transform type from among a plurality of transform types, and transform prediction errors into transform coefficients by using a transform basis function corresponding to the selected transform type. This sort of transform is also referred to as explicit multiple core transform (EMT) or adaptive multiple transform (AMT).

The transform types include, for example, DCT-II, DCT-V, DCT-VIII, DST-I, and DST-VII. FIG. 3 is a chart indicating transform basis functions for each transform type. In FIG. 3, N indicates the number of input pixels. For example, selection of a transform type from among the plurality of transform types may depend on the prediction type (intra prediction and inter prediction), and may depend on intra prediction mode.

Information indicating whether to apply such EMT or AMT (referred to as, for example, an AMT flag) and information indicating the selected transform type is signaled at the CU level. Note that the signaling of such information need not be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, or CTU level).

Moreover, transformer **106** may apply a secondary transform to the transform coefficients (transform result). Such a secondary transform is also referred to as adaptive secondary transform (AST) or non-separable secondary transform (NSST). For example, transformer **106** applies a secondary transform to each sub-block (for example, each 4×4 sub-block) included in the block of the transform coefficients

corresponding to the intra prediction errors. Information indicating whether to apply NSST and information related to the transform matrix used in NSST are signalled at the CU level. Note that the signaling of such information need not be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, or CTU level).

Here, a separable transform is a method in which a transform is performed a plurality of times by separately performing a transform for each direction according to the number of dimensions input. A non-separable transform is a method of performing a collective transform in which two or more dimensions in a multidimensional input are collectively regarded as a single dimension.

In one example of a non-separable transform, when the input is a 4×4 block, the 4×4 block is regarded as a single array including 16 components, and the transform applies a 16×16 transform matrix to the array.

Moreover, similar to above, after an input 4×4 block is regarded as a single array including 16 components, a transform that performs a plurality of Givens rotations on the array (i.e., a Hypercube-Givens Transform) is also one example of a non-separable transform.

Quantizer

Quantizer **108** quantizes the transform coefficients output from transformer **106**. More specifically, quantizer **108** scans, in a predetermined scanning order, the transform coefficients of the current block, and quantizes the scanned transform coefficients based on quantization parameters (QP) corresponding to the transform coefficients. Quantizer **108** then outputs the quantized transform coefficients (hereinafter referred to as quantized coefficients) of the current block to entropy encoder **110** and inverse quantizer **112**.

A predetermined order is an order for quantizing/inverse quantizing transform coefficients. For example, a predetermined scanning order is defined as ascending order of frequency (from low to high frequency) or descending order of frequency (from high to low frequency).

A quantization parameter is a parameter defining a quantization step size (quantization width). For example, if the value of the quantization parameter increases, the quantization step size also increases. In other words, if the value of the quantization parameter increases, the quantization error increases.

Entropy Encoder

Entropy encoder **110** generates an encoded signal (encoded bitstream) by variable length encoding quantized coefficients, which are inputs from quantizer **108**. More specifically, entropy encoder **110**, for example, binarizes quantized coefficients and arithmetic encodes the binary signal.

Inverse Quantizer

Inverse quantizer **112** inverse quantizes quantized coefficients, which are inputs from quantizer **108**. More specifically, inverse quantizer **112** inverse quantizes, in a predetermined scanning order, quantized coefficients of the current block. Inverse quantizer **112** then outputs the inverse quantized transform coefficients of the current block to inverse transformer **114**.

Inverse Transformer

Inverse transformer **114** restores prediction errors by inverse transforming transform coefficients, which are

inputs from inverse quantizer **112**. More specifically, inverse transformer **114** restores the prediction errors of the current block by applying an inverse transform corresponding to the transform applied by transformer **106** on the transform coefficients. Inverse transformer **114** then outputs the restored prediction errors to adder **116**.

Note that since information is lost in quantization, the restored prediction errors do not match the prediction errors calculated by subtractor **104**. In other words, the restored prediction errors include quantization errors.

Adder

Adder **116** reconstructs the current block by summing prediction errors, which are inputs from inverse transformer **114**, and prediction samples, which are inputs from prediction controller **128**. Adder **116** then outputs the reconstructed block to block memory **118** and loop filter **120**. A reconstructed block is also referred to as a local decoded block.

Block Memory

Block memory **118** is storage for storing blocks in a picture to be encoded (hereinafter referred to as a current picture) for reference in intra prediction. More specifically, block memory **118** stores reconstructed blocks output from adder **116**.

Loop Filter

Loop filter **120** applies a loop filter to blocks reconstructed by adder **116**, and outputs the filtered reconstructed blocks to frame memory **122**. A loop filter is a filter used in an encoding loop (in-loop filter), and includes, for example, a deblocking filter (DF), a sample adaptive offset (SAO), and an adaptive loop filter (ALF).

In ALF, a least square error filter for removing compression artifacts is applied. For example, one filter from among a plurality of filters is selected for each 2×2 sub-block in the current block based on direction and activity of local gradients, and is applied.

More specifically, first, each sub-block (for example, each 2×2 sub-block) is categorized into one out of a plurality of classes (for example, 15 or 25 classes). The classification of the sub-block is based on gradient directionality and activity. For example, classification index C is derived based on gradient directionality D (for example, 0 to 2 or 0 to 4) and gradient activity A (for example, 0 to 4) (for example, C=5D+A). Then, based on classification index C, each sub-block is categorized into one out of a plurality of classes (for example, 15 or 25 classes).

For example, gradient directionality D is calculated by comparing gradients of a plurality of directions (for example, the horizontal, vertical, and two diagonal directions). Moreover, for example, gradient activity A is calculated by summing gradients of a plurality of directions and quantizing the sum.

The filter to be used for each sub-block is determined from among the plurality of filters based on the result of such categorization.

The filter shape to be used in ALF is, for example, a circular symmetric filter shape. FIG. 4A through FIG. 4C illustrate examples of filter shapes used in ALF. FIG. 4A illustrates a 5×5 diamond shape filter, FIG. 4B illustrates a 7×7 diamond shape filter, and FIG. 4C illustrates a 9×9 diamond shape filter. Information indicating the filter shape is signalled at the picture level. Note that the signaling of

11

information indicating the filter shape need not be performed at the picture level, and may be performed at another level (for example, at the sequence level, slice level, tile level, CTU level, or CU level).

The enabling or disabling of ALF is determined at the picture level or CU level. For example, for luma, the decision to apply ALF or not is done at the CU level, and for chroma, the decision to apply ALF or not is done at the picture level. Information indicating whether ALF is enabled or disabled is signalled at the picture level or CU level. Note that the signaling of information indicating whether ALF is enabled or disabled need not be performed at the picture level or CU level, and may be performed at another level (for example, at the sequence level, slice level, tile level, or CTU level).

The coefficients set for the plurality of selectable filters (for example, 15 or 25 filters) is signalled at the picture level. Note that the signaling of the coefficients set need not be performed at the picture level, and may be performed at another level (for example, at the sequence level, slice level, tile level, CTU level, CU level, or sub-block level).

Frame Memory

Frame memory **122** is storage for storing reference pictures used in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory **122** stores reconstructed blocks filtered by loop filter **120**.

Intra Predictor

Intra predictor **124** generates a prediction signal (intra prediction signal) by intra predicting the current block with reference to a block or blocks in the current picture and stored in block memory **118** (also referred to as intra frame prediction). More specifically, intra predictor **124** generates an intra prediction signal by intra prediction with reference to samples (for example, luma and/or chroma values) of a block or blocks neighboring the current block, and then outputs the intra prediction signal to prediction controller **128**.

For example, intra predictor **124** performs intra prediction by using one mode from among a plurality of predefined intra prediction modes. The intra prediction modes include one or more non-directional prediction modes and a plurality of directional prediction modes.

The one or more non-directional prediction modes include, for example, planar prediction mode and DC prediction mode defined in the H.265/high-efficiency video coding (HEVC) standard (see H.265(ISO/IEC 23008-2 HEVC(High Efficiency Video Coding))).

The plurality of directional prediction modes include, for example, the 33 directional prediction modes defined in the H.265/HEVC standard. Note that the plurality of directional prediction modes may further include 32 directional prediction modes in addition to the 33 directional prediction modes (for a total of 65 directional prediction modes). FIG. 5A illustrates 67 intra prediction modes used in intra prediction (two non-directional prediction modes and 65 directional prediction modes). The solid arrows represent the 33 directions defined in the H.265/HEVC standard, and the dashed arrows represent the additional 32 directions.

Note that a luma block may be referenced in chroma block intra prediction. In other words, a chroma component of the current block may be predicted based on a luma component of the current block. Such intra prediction is also referred to as cross-component linear model (CCLM) prediction. Such

12

a chroma block intra prediction mode that references a luma block (referred to as, for example, CCLM mode) may be added as one of the chroma block intra prediction modes.

Intra predictor **124** may correct post-intra-prediction pixel values based on horizontal/vertical reference pixel gradients. Intra prediction accompanied by this sort of correcting is also referred to as position dependent intra prediction combination (PDPC). Information indicating whether to apply PDPC or not (referred to as, for example, a PDPC flag) is, for example, signalled at the CU level. Note that the signaling of this information need not be performed at the CU level, and may be performed at another level (for example, on the sequence level, picture level, slice level, tile level, or CTU level).

Inter Predictor

Inter predictor **126** generates a prediction signal (inter prediction signal) by inter predicting the current block with reference to a block or blocks in a reference picture, which is different from the current picture and is stored in frame memory **122** (also referred to as inter frame prediction). Inter prediction is performed per current block or per sub-block (for example, per 4×4 block) in the current block. For example, inter predictor **126** performs motion estimation in a reference picture for the current block or sub-block. Inter predictor **126** then generates an inter prediction signal of the current block or sub-block by motion compensation by using motion information (for example, a motion vector) obtained from motion estimation. Inter predictor **126** then outputs the generated inter prediction signal to prediction controller **128**.

The motion information used in motion compensation is signalled. A motion vector predictor may be used for the signaling of the motion vector. In other words, the difference between the motion vector and the motion vector predictor may be signalled.

Note that the inter prediction signal may be generated using motion information for a neighboring block in addition to motion information for the current block obtained from motion estimation. More specifically, the inter prediction signal may be generated per sub-block in the current block by calculating a weighted sum of a prediction signal based on motion information obtained from motion estimation and a prediction signal based on motion information for a neighboring block. Such inter prediction (motion compensation) is also referred to as overlapped block motion compensation (OBMC).

In such an OBMC mode, information indicating sub-block size for OBMC (referred to as, for example, OBMC block size) is signalled at the sequence level. Moreover, information indicating whether to apply the OBMC mode or not (referred to as, for example, an OBMC flag) is signalled at the CU level. Note that the signaling of such information need not be performed at the sequence level and CU level, and may be performed at another level (for example, at the picture level, slice level, tile level, CTU level, or sub-block level).

Hereinafter, the OBMC mode will be described in further detail. FIG. 5B is a flowchart and FIG. 5C is a conceptual diagram for illustrating an outline of a prediction image correction process performed via OBMC processing.

First, a prediction image (Pred) is obtained through typical motion compensation using a motion vector (MV) assigned to the current block.

Next, a prediction image (Pred_L) is obtained by applying a motion vector (MV_L) of the encoded neighboring left

block to the current block, and a first pass of the correction of the prediction image is made by superimposing the prediction image and Pred_L.

Similarly, a prediction image (Pred_U) is obtained by applying a motion vector (MV_U) of the encoded neighboring upper block to the current block, and a second pass of the correction of the prediction image is made by superimposing the prediction image resulting from the first pass and Pred_U. The result of the second pass is the final prediction image.

Note that the above example is of a two-pass correction method using the neighboring left and upper blocks, but the method may be a three-pass or higher correction method that also uses the neighboring right and/or lower block.

Note that the region subject to superimposition may be the entire pixel region of the block, and, alternatively, may be a partial block boundary region.

Note that here, the prediction image correction process is described as being based on a single reference picture, but the same applies when a prediction image is corrected based on a plurality of reference pictures. In such a case, after corrected prediction images resulting from performing correction based on each of the reference pictures are obtained, the obtained corrected prediction images are further superimposed to obtain the final prediction image.

Note that the unit of the current block may be a prediction block and, alternatively, may be a sub-block obtained by further dividing the prediction block.

One example of a method for determining whether to implement OBMC processing is by using an obmc_flag, which is a signal that indicates whether to implement OBMC processing. As one specific example, the encoder determines whether the current block belongs to a region including complicated motion. The encoder sets the obmc_flag to a value of "1" when the block belongs to a region including complicated motion and implements OBMC processing when encoding, and sets the obmc_flag to a value of "0" when the block does not belong to a region including complication motion and encodes without implementing OBMC processing. The decoder switches between implementing OBMC processing or not by decoding the obmc_flag written in the stream and performing the decoding in accordance with the flag value.

Note that the motion information may be derived on the decoder side without being signalled. For example, a merge mode defined in the H.265/HEVC standard may be used. Moreover, for example, the motion information may be derived by performing motion estimation on the decoder side. In this case, motion estimation is performed without using the pixel values of the current block.

Here, a mode for performing motion estimation on the decoder side will be described. A mode for performing motion estimation on the decoder side is also referred to as pattern matched motion vector derivation (PMMVD) mode or frame rate up-conversion (FRUC) mode.

One example of FRUC processing is illustrated in FIG. 5D. First, a candidate list (a candidate list may be a merge list) of candidates each including a motion vector predictor is generated with reference to motion vectors of encoded blocks that spatially or temporally neighbor the current block. Next, the best candidate MV is selected from among a plurality of candidate MVs registered in the candidate list. For example, evaluation values for the candidates included in the candidate list are calculated and one candidate is selected based on the calculated evaluation values.

Next, a motion vector for the current block is derived from the motion vector of the selected candidate. More

specifically, for example, the motion vector for the current block is calculated as the motion vector of the selected candidate (best candidate MV), as-is. Alternatively, the motion vector for the current block may be derived by pattern matching performed in the vicinity of a position in a reference picture corresponding to the motion vector of the selected candidate. In other words, when the vicinity of the best candidate MV is searched via the same method and an MV having a better evaluation value is found, the best candidate MV may be updated to the MV having the better evaluation value, and the MV having the better evaluation value may be used as the final MV for the current block. Note that a configuration in which this processing is not implemented is also acceptable.

The same processes may be performed in cases in which the processing is performed in units of sub-blocks.

Note that an evaluation value is calculated by calculating the difference in the reconstructed image by pattern matching performed between a region in a reference picture corresponding to a motion vector and a predetermined region. Note that the evaluation value may be calculated by using some other information in addition to the difference.

The pattern matching used is either first pattern matching or second pattern matching. First pattern matching and second pattern matching are also referred to as bilateral matching and template matching, respectively.

In the first pattern matching, pattern matching is performed between two blocks along the motion trajectory of the current block in two different reference pictures. Therefore, in the first pattern matching, a region in another reference picture conforming to the motion trajectory of the current block is used as the predetermined region for the above-described calculation of the candidate evaluation value.

FIG. 6 is for illustrating one example of pattern matching (bilateral matching) between two blocks along a motion trajectory. As illustrated in FIG. 6, in the first pattern matching, two motion vectors (MV0, MV1) are derived by finding the best match between two blocks along the motion trajectory of the current block (Cur block) in two different reference pictures (Ref0, Ref1). More specifically, a difference between (i) a reconstructed image in a specified position in a first encoded reference picture (Ref0) specified by a candidate MV and (ii) a reconstructed picture in a specified position in a second encoded reference picture (Ref1) specified by a symmetrical MV scaled at a display time interval of the candidate MV may be derived, and the evaluation value for the current block may be calculated by using the derived difference. The candidate MV having the best evaluation value among the plurality of candidate MVs may be selected as the final MV.

Under the assumption of continuous motion trajectory, the motion vectors (MV0, MV1) pointing to the two reference blocks shall be proportional to the temporal distances (TD0, TD1) between the current picture (Cur Pic) and the two reference pictures (Ref0, Ref1). For example, when the current picture is temporally between the two reference pictures and the temporal distance from the current picture to the two reference pictures is the same, the first pattern matching derives a mirror based bi-directional motion vector.

In the second pattern matching, pattern matching is performed between a template in the current picture (blocks neighboring the current block in the current picture (for example, the top and/or left neighboring blocks)) and a block in a reference picture. Therefore, in the second pattern matching, a block neighboring the current block in the

15

current picture is used as the predetermined region for the above-described calculation of the candidate evaluation value.

FIG. 7 is for illustrating one example of pattern matching (template matching) between a template in the current picture and a block in a reference picture. As illustrated in FIG. 7, in the second pattern matching, a motion vector of the current block is derived by searching a reference picture (Ref0) to find the block that best matches neighboring blocks of the current block (Cur block) in the current picture (Cur Pic). More specifically, a difference between (i) a reconstructed image of an encoded region that is both or one of the neighboring left and neighboring upper region and (ii) a reconstructed picture in the same position in an encoded reference picture (Ref0) specified by a candidate MV may be derived, and the evaluation value for the current block may be calculated by using the derived difference. The candidate MV having the best evaluation value among the plurality of candidate MVs may be selected as the best candidate MV.

Information indicating whether to apply the FRUC mode or not (referred to as, for example, a FRUC flag) is signalled at the CU level. Moreover, when the FRUC mode is applied (for example, when the FRUC flag is set to true), information indicating the pattern matching method (first pattern matching or second pattern matching) is signalled at the CU level. Note that the signaling of such information need not be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, CTU level, or sub-block level).

Here, a mode for deriving a motion vector based on a model assuming uniform linear motion will be described. This mode is also referred to as a bi-directional optical flow (BIO) mode.

FIG. 8 is for illustrating a model assuming uniform linear motion. In FIG. 8, (v_x, v_y) denotes a velocity vector, and τ_0 and τ_1 denote temporal distances between the current picture (Cur Pic) and two reference pictures (Ref₀, Ref₁). (MV_{x_0}, MV_{y_0}) denotes a motion vector corresponding to reference picture Ref₀, and (MV_{x_1}, MV_{y_1}) denotes a motion vector corresponding to reference picture Ref₁.

Here, under the assumption of uniform linear motion exhibited by velocity vector (v_x, v_y) , (MV_{x_0}, MV_{y_0}) and (MV_{x_1}, MV_{y_1}) are represented as $(v_x\tau_0, v_y\tau_0)$ and $(-v_x\tau_1, -v_y\tau_1)$, respectively, and the following optical flow equation is given.

MATH. 1

$$\partial I^{(k)} / \partial t + v_x \partial I^{(k)} / \partial x + v_y \partial I^{(k)} / \partial y = 0. \quad (1)$$

Here, $I^{(k)}$ denotes a luma value from reference picture k ($k=0, 1$) after motion compensation. This optical flow equation shows that the sum of (i) the time derivative of the luma value, (ii) the product of the horizontal velocity and the horizontal component of the spatial gradient of a reference picture, and (iii) the product of the vertical velocity and the vertical component of the spatial gradient of a reference picture is equal to zero. A motion vector of each block obtained from, for example, a merge list is corrected pixel by pixel based on a combination of the optical flow equation and Hermite interpolation.

Note that a motion vector may be derived on the decoder side using a method other than deriving a motion vector based on a model assuming uniform linear motion. For

16

example, a motion vector may be derived for each sub-block based on motion vectors of neighboring blocks.

Here, a mode in which a motion vector is derived for each sub-block based on motion vectors of neighboring blocks will be described. This mode is also referred to as affine motion compensation prediction mode.

FIG. 9A is for illustrating deriving a motion vector of each sub-block based on motion vectors of neighboring blocks. In FIG. 9A, the current block includes 16 4x4 sub-blocks. Here, motion vector v_0 of the top left corner control point in the current block is derived based on motion vectors of neighboring sub-blocks, and motion vector v_1 of the top right corner control point in the current block is derived based on motion vectors of neighboring blocks. Then, using the two motion vectors v_0 and v_1 , the motion vector (v_x, v_y) of each sub-block in the current block is derived using Equation 2 below.

MATH. 2

$$\begin{cases} v_x = \frac{(v_{1x} - v_{0x})}{w}x - \frac{(v_{1y} - v_{0y})}{w}y + v_{0x} \\ v_y = \frac{(v_{1y} - v_{0y})}{w}x + \frac{(v_{1x} - v_{0x})}{w}y + v_{0y} \end{cases} \quad (2)$$

Here, x and y are the horizontal and vertical positions of the sub-block, respectively, and w is a predetermined weighted coefficient.

Such an affine motion compensation prediction mode may include a number of modes of different methods of deriving the motion vectors of the top left and top right corner control points. Information indicating such an affine motion compensation prediction mode (referred to as, for example, an affine flag) is signalled at the CU level. Note that the signaling of information indicating the affine motion compensation prediction mode need not be performed at the CU level, and may be performed at another level (for example, at the sequence level, picture level, slice level, tile level, CTU level, or sub-block level).

Prediction Controller

Prediction controller 128 selects either the intra prediction signal or the inter prediction signal, and outputs the selected prediction signal to subtractor 104 and adder 116.

Here, an example of deriving a motion vector via merge mode in a current picture will be given. FIG. 9B is for illustrating an outline of a process for deriving a motion vector via merge mode.

First, an MV predictor list in which candidate MV predictors are registered is generated. Examples of candidate MV predictors include: spatially neighboring MV predictors, which are MVs of encoded blocks positioned in the spatial vicinity of the current block; a temporally neighboring MV predictor, which is an MV of a block in an encoded reference picture that neighbors a block in the same location as the current block; a combined MV predictor, which is an MV generated by combining the MV values of the spatially neighboring MV predictor and the temporally neighboring MV predictor; and a zero MV predictor, which is an MV whose value is zero.

Next, the MV of the current block is determined by selecting one MV predictor from among the plurality of MV predictors registered in the MV predictor list.

Furthermore, in the variable-length encoder, a `merge_idx`, which is a signal indicating which MV predictor is selected, is written and encoded into the stream.

Note that the MV predictors registered in the MV predictor list illustrated in FIG. 9B constitute one example. The number of MV predictors registered in the MV predictor list may be different from the number illustrated in FIG. 9B, the MV predictors registered in the MV predictor list may omit one or more of the types of MV predictors given in the example in FIG. 9B, and the MV predictors registered in the MV predictor list may include one or more types of MV predictors in addition to and different from the types given in the example in FIG. 9B.

Note that the final MV may be determined by performing DMVR processing (to be described later) by using the MV of the current block derived via merge mode.

Here, an example of determining an MV by using DMVR processing will be given.

FIG. 9C is a conceptual diagram for illustrating an outline of DMVR processing.

First, the most appropriate MVP set for the current block is considered to be the candidate MV, reference pixels are obtained from a first reference picture, which is a picture processed in the L0 direction in accordance with the candidate MV, and a second reference picture, which is a picture processed in the L1 direction in accordance with the candidate MV, and a template is generated by calculating the average of the reference pixels.

Next, using the template, the surrounding regions of the candidate MVs of the first and second reference pictures are searched, and the MV with the lowest cost is determined to be the final MV. Note that the cost value is calculated using, for example, the difference between each pixel value in the template and each pixel value in the regions searched, as well as the MV value.

Note that the outlines of the processes described here are fundamentally the same in both the encoder and the decoder.

Note that processing other than the processing exactly as described above may be used, so long as the processing is capable of deriving the final MV by searching the surroundings of the candidate MV.

Here, an example of a mode that generates a prediction image by using LIC processing will be given.

FIG. 9D is for illustrating an outline of a prediction image generation method using a luminance correction process performed via LIC processing.

First, an MV is extracted for obtaining, from an encoded reference picture, a reference image corresponding to the current block.

Next, information indicating how the luminance value changed between the reference picture and the current picture is extracted and a luminance correction parameter is calculated by using the luminance pixel values for the encoded left neighboring reference region and the encoded upper neighboring reference region, and the luminance pixel value in the same location in the reference picture specified by the MV.

The prediction image for the current block is generated by performing a luminance correction process by using the luminance correction parameter on the reference image in the reference picture specified by the MV.

Note that the shape of the surrounding reference region illustrated in FIG. 9D is just one example; the surrounding reference region may have a different shape.

Moreover, although a prediction image is generated from a single reference picture in this example, in cases in which a prediction image is generated from a plurality of reference

pictures as well, the prediction image is generated after performing a luminance correction process, via the same method, on the reference images obtained from the reference pictures.

One example of a method for determining whether to implement LIC processing is by using an `lic_flag`, which is a signal that indicates whether to implement LIC processing. As one specific example, the encoder determines whether the current block belongs to a region of luminance change. The encoder sets the `lic_flag` to a value of "1" when the block belongs to a region of luminance change and implements LIC processing when encoding, and sets the `lic_flag` to a value of "0" when the block does not belong to a region of luminance change and encodes without implementing LIC processing. The decoder switches between implementing LIC processing or not by decoding the `lic_flag` written in the stream and performing the decoding in accordance with the flag value.

One example of a different method of determining whether to implement LIC processing is determining so in accordance with whether LIC processing was determined to be implemented for a surrounding block. In one specific example, when merge mode is used on the current block, whether LIC processing was applied in the encoding of the surrounding encoded block selected upon deriving the MV in the merge mode processing may be determined, and whether to implement LIC processing or not can be switched based on the result of the determination. Note that in this example, the same applies to the processing performed on the decoder side.

Decoder Outline

Next, a decoder capable of decoding an encoded signal (encoded bitstream) output from encoder 100 will be described. FIG. 10 is a block diagram illustrating a functional configuration of decoder 200 according to Embodiment 1. Decoder 200 is a moving picture/picture decoder that decodes a moving picture/picture block by block.

As illustrated in FIG. 10, decoder 200 includes entropy decoder 202, inverse quantizer 204, inverse transformer 206, adder 208, block memory 210, loop filter 212, frame memory 214, intra predictor 216, inter predictor 218, and prediction controller 220.

Decoder 200 is realized as, for example, a generic processor and memory. In this case, when a software program stored in the memory is executed by the processor, the processor functions as entropy decoder 202, inverse quantizer 204, inverse transformer 206, adder 208, loop filter 212, intra predictor 216, inter predictor 218, and prediction controller 220. Alternatively, decoder 200 may be realized as one or more dedicated electronic circuits corresponding to entropy decoder 202, inverse quantizer 204, inverse transformer 206, adder 208, loop filter 212, intra predictor 216, inter predictor 218, and prediction controller 220.

Hereinafter, each component included in decoder 200 will be described.

Entropy Decoder

Entropy decoder 202 entropy decodes an encoded bitstream. More specifically, for example, entropy decoder 202 arithmetic decodes an encoded bitstream into a binary signal. Entropy decoder 202 then debinarizes the binary signal. With this, entropy decoder 202 outputs quantized coefficients of each block to inverse quantizer 204.

19

Inverse Quantizer

Inverse quantizer **204** inverse quantizes quantized coefficients of a block to be decoded (hereinafter referred to as a current block), which are inputs from entropy decoder **202**. More specifically, inverse quantizer **204** inverse quantizes quantized coefficients of the current block based on quantization parameters corresponding to the quantized coefficients. Inverse quantizer **204** then outputs the inverse quantized coefficients (i.e., transform coefficients) of the current block to inverse transformer **206**.

Inverse Transformer

Inverse transformer **206** restores prediction errors by inverse transforming transform coefficients, which are inputs from inverse quantizer **204**.

For example, when information parsed from an encoded bitstream indicates application of EMT or AMT (for example, when the AMT flag is set to true), inverse transformer **206** inverse transforms the transform coefficients of the current block based on information indicating the parsed transform type.

Moreover, for example, when information parsed from an encoded bitstream indicates application of NSST, inverse transformer **206** applies a secondary inverse transform to the transform coefficients.

Adder

Adder **208** reconstructs the current block by summing prediction errors, which are inputs from inverse transformer **206**, and prediction samples, which is an input from prediction controller **220**. Adder **208** then outputs the reconstructed block to block memory **210** and loop filter **212**.

Block Memory

Block memory **210** is storage for storing blocks in a picture to be decoded (hereinafter referred to as a current picture) for reference in intra prediction. More specifically, block memory **210** stores reconstructed blocks output from adder **208**.

Loop Filter

Loop filter **212** applies a loop filter to blocks reconstructed by adder **208**, and outputs the filtered reconstructed blocks to frame memory **214** and, for example, a display device.

When information indicating the enabling or disabling of ALF parsed from an encoded bitstream indicates enabled, one filter from among a plurality of filters is selected based on direction and activity of local gradients, and the selected filter is applied to the reconstructed block.

Frame Memory

Frame memory **214** is storage for storing reference pictures used in inter prediction, and is also referred to as a frame buffer. More specifically, frame memory **214** stores reconstructed blocks filtered by loop filter **212**.

Intra Predictor

Intra predictor **216** generates a prediction signal (intra prediction signal) by intra prediction with reference to a

20

block or blocks in the current picture and stored in block memory **210**. More specifically, intra predictor **216** generates an intra prediction signal by intra prediction with reference to samples (for example, luma and/or chroma values) of a block or blocks neighboring the current block, and then outputs the intra prediction signal to prediction controller **220**.

Note that when an intra prediction mode in which a chroma block is intra predicted from a luma block is selected, intra predictor **216** may predict the chroma component of the current block based on the luma component of the current block.

Moreover, when information indicating the application of PDPC is parsed from an encoded bitstream, intra predictor **216** corrects post-intra-prediction pixel values based on horizontal/vertical reference pixel gradients.

Inter Predictor

Inter predictor **218** predicts the current block with reference to a reference picture stored in frame memory **214**. Inter prediction is performed per current block or per sub-block (for example, per 4×4 block) in the current block. For example, inter predictor **218** generates an inter prediction signal of the current block or sub-block by motion compensation by using motion information (for example, a motion vector) parsed from an encoded bitstream, and outputs the inter prediction signal to prediction controller **220**.

Note that when the information parsed from the encoded bitstream indicates application of OBMC mode, inter predictor **218** generates the inter prediction signal using motion information for a neighboring block in addition to motion information for the current block obtained from motion estimation.

Moreover, when the information parsed from the encoded bitstream indicates application of FRUC mode, inter predictor **218** derives motion information by performing motion estimation in accordance with the pattern matching method (bilateral matching or template matching) parsed from the encoded bitstream. Inter predictor **218** then performs motion compensation using the derived motion information.

Moreover, when BIO mode is to be applied, inter predictor **218** derives a motion vector based on a model assuming uniform linear motion. Moreover, when the information parsed from the encoded bitstream indicates that affine motion compensation prediction mode is to be applied, inter predictor **218** derives a motion vector of each sub-block based on motion vectors of neighboring blocks.

Prediction Controller

Prediction controller **220** selects either the intra prediction signal or the inter prediction signal, and outputs the selected prediction signal to adder **208**.

Aspect 1 of Inter Prediction

In aspect 1 through aspect 5 of the present embodiment, inter prediction performed in encoder **100** and decoder **200** by introducing a process for deriving a motion vector will be described. Note that this technique is referred to as Ultimate Motion Vector Expression (UMVE), but it is also referred to as Merge mode Motion Vector Difference (MMVD).

The following primarily describes the operation of decoder **200**, but the operation of encoder **100** is also the same.

21

FIG. 11 is a flow chart illustrating an example of the internal processing of decoder 200 according to aspect 1 of Embodiment 1. FIG. 12 is for illustrating a delta motion vector used in inter prediction according to aspect 1 of Embodiment 1.

Firstly, as shown in FIG. 11, inter predictor 218 in decoder 200 parses a delta motion vector representing (i) either one of the first direction or the second direction which is perpendicular to the other one and (ii) a distance from a base motion vector (S1001). More specifically, inter predictor 218 parses information obtained from an encoded bitstream to determine the base motion vector and parse the delta motion vector.

Here, the delta motion vector is additional information for the base motion vector, and extended information of the base motion vector. More specifically, the delta motion vector represents a motion vector difference from the base motion vector in one of multiple directions including diagonal directions, and is defined by a direction and a magnitude (step). In the present aspect, the delta motion vector is information represented by (i) either one of the first direction or the second direction which is perpendicular to the other one, among the predetermined multiple directions, and (ii) a distance from the base motion vector.

Moreover, the base motion vector is, for example, one MV predictor selected from a MV predictor list in merge mode. In other words, the inter prediction according to the present aspect may be inter prediction in merge mode. Inter predictor 218 may determine the base motion vector by selecting one candidate motion vector from a list indicating candidate motion vectors for a current block (the MV predictor list).

In the example shown in FIG. 12, the base motion vector is assumed to be located at the origin of the x-axis and the y-axis, and the first direction or the second direction which is diagonal corresponds to a direction inclined by 45 degrees to the x- or y-axis direction. In the example shown in FIG. 12, the first direction includes not only the right diagonal direction in the first quadrant (+45°) but also a direction extending in the third quadrant (+225°). Similarly, the second direction includes not only the left diagonal direction in the fourth quadrant (-45°) but also a direction extending in the second quadrant (-225°). Moreover, as shown in FIG. 12, the first direction and the second direction are perpendicular to each other, and the first direction is along the diagonal line between the x-axis and the y-axis. The delta motion vector is along either one of the positive side or the negative side of the first direction or the second direction, and is represented as a vector having a magnitude (step) in a position denoted by a circle.

Moreover, in the present aspect, inter predictor 218 can parse a direction parameter and a magnitude parameter to obtain a direction and a magnitude defining the delta motion vector from the predetermined list indicating multiple directions and multiple magnitudes. Note that the predetermined list indicates multiple directions and multiple magnitudes when it is assumed that the base motion vector is located at the origin of the x-axis and the y-axis. In the direction parameter, a different value may be assigned to the positive side of the first direction, the negative side of the first direction, the positive side of the second direction, and the negative side of the second direction.

Next, as shown in FIG. 11, inter predictor 218 decodes the current block using at least the delta motion vector parsed at step S1001 (S1002). More specifically, inter predictor 218 decodes the current block, i.e., generates a prediction signal

22

of the current block, by performing motion compensation using the delta motion vector parsed at step S1001 and the base motion vector.

Note that in the above description, for example, a current block of uni-directional prediction mode is decoded such that one MV predictor selected from the MV predictor list in which MVs of decoded blocks in a forward reference picture are registered as candidate MV predictors is determined as the reference vector, but the present disclosure is not limited to this.

A current block of bi-directional prediction mode may be decoded. In this case, when the above delta motion vector is determined as the first delta motion vector, the current block of bi-directional prediction mode may be decoded using the second delta motion vector which is symmetric to the first delta motion vector with respect to the origin. For example, assuming that the first delta motion vector for the first reference picture in the L0 direction is $[i, j]$, the second delta motion vector for the second reference picture in the L1 direction is $[-i, -j]$.

Aspect 2 of Inter Prediction

In the present aspect, inter predictor 218 in decoder 200 performs a parsing process of the delta motion vector in two steps. The following mainly describes the differences from aspect 1.

FIG. 13 is a flow chart illustrating an example of the internal processing of decoder 200 according to aspect 2 of Embodiment 1. FIG. 14 is for illustrating a delta motion vector used in inter prediction according to aspect 2 of Embodiment 1.

First, as shown in FIG. 13, inter predictor 218 in decoder 200 parses a parameter such as a prediction parameter, and selects a set of directions perpendicular to each other including a diagonal direction or a set of directions perpendicular to each other including a horizontal direction, from among the predetermined multiple directions (S1001).

Here, this step is specifically described with reference to FIG. 14. As the first step, inter predictor 218 parses a parameter such as a prediction parameter obtained from information obtained from an encoded bitstream, to select two directions from among the four directions shown in FIG. 14. In FIG. 14, as the four directions, the first direction, the second direction, the third direction, and the fourth direction are indicated. Note that in the example shown in FIG. 14, the first direction includes not only the right diagonal direction in the first quadrant (+45°) but also a direction extending in the third quadrant (+225°). Similarly, the second direction includes not only the left diagonal direction in the fourth quadrant (-45°) but also a direction extending in the second quadrant (-225°). The third direction is indicated as a horizontal direction including the positive side and the negative side, and the fourth direction is indicated as a vertical direction including the positive side and the negative side.

As described above, in the present aspect, inter predictor 218 selects a set of the first direction and the second direction which are perpendicular to each other from among the predetermined multiple directions, based on the obtained prediction parameter.

Next, as shown in FIG. 13, inter predictor 218 selects one direction from the set selected at step S1001, and parses the delta motion vector representing the selected one direction and a distance from the base motion vector (S1002). For example, inter predictor 218 can select one direction from the set selected at step S1001 by parsing a direction index

23

and a magnitude index. As described above, inter predictor **218** selects, as the one direction, either one of the first direction or the second direction of the set selected at step **S10011**.

Here, this step is specifically described with reference to FIG. **14**. As the second step, inter predictor **218** parses values indicated by the direction index and the magnitude index included in the parameter obtained from the information obtained from the encoded bitstream, to select one direction and either one of the positive side or negative side from among the selected two directions shown in FIG. **14** and determine the magnitude in the selected one direction. In this way, inter predictor **218** can obtain the delta motion vector representing the selected one direction and a distance from the base motion vector.

Aspect 3 of Inter Prediction

FIG. **15** is a flow chart illustrating an example of the internal processing of decoder **200** according to aspect 3 of Embodiment 1. FIG. **16** is a diagram illustrating an example of a MV predictor list according to aspect 3 of Embodiment 1. FIG. **17** is for illustrating a delta motion vector used in inter prediction according to aspect 3 of Embodiment 1. The following mainly describes the differences from aspects 1 and 2.

First, as shown in FIG. **15**, inter predictor **218** in decoder **200** selects the first motion vector from a list of motion vectors (**S10013**). More specifically, inter predictor **218** selects the first motion vector as the base motion vector from a list indicating candidate motion vectors for a current block.

Here, the list of motion vectors, i.e., a list indicating candidate motion vectors for the current block, is a MV predictor list used in inter prediction in merge mode, for example. In this case, inter predictor **218** can select the first motion vector as shown in (a) of FIG. **17** by parsing a parameter indicating a candidate index (candidate IDX) as shown in FIG. **16**.

Next, inter predictor **218** derives the first direction and the second direction which are perpendicular to each other using at least the direction of the first motion vector selected at step **S10013** (**S10014**). For example, as shown in (b) of FIG. **17**, an example of the derived first direction is substantially the same as the direction of the first motion vector, and the second direction is perpendicular to the first direction. In other words, inter predictor **218** derives the first direction substantially the same as the direction of the first motion vector using the direction of the first motion vector selected at step **S10013** to derive the first direction and the second direction which are perpendicular to each other. An example of the details of this deriving process will be described below.

Next, inter predictor **218** parses the delta motion vector representing either one of the first direction or the second direction and a distance from the first motion vector (**S10015**). In the present aspect, inter predictor **218** determines either one of the first direction or the second direction as the one direction, and parses the delta motion vector representing (i) the derived one direction and either one of its positive side or negative side, and (ii) a distance from the first motion vector. The delta motion vector is defined by the direction index and the magnitude index.

The following describes an example of the details of the deriving process of the direction substantially the same as the first motion vector, with reference to FIG. **17** through FIG. **19**. FIG. **18** is a diagram illustrating an example of a look up table according to aspect 3 of Embodiment 1. FIG.

24

19 is a table illustrating an example of a relationship between magnitude indexes and shift values in accordance with pixel precision according to aspect 3 of Embodiment 1.

First, at the first step of the deriving process, inter predictor **218** determines a minimum value denoted as "min" and a maximum value denoted as "max" using component values (x value and y value) of the selected first motion vector. Here, in an example shown in (a) of FIG. **17**, the component values of the selected first motion vector are [m, n]. For example, when the component values of the first motion vector are [100, 64], "min"=64 and "max"=100 are true.

Subsequently, at the second step of the deriving process, inter predictor **218** derives the first value, which is an integer, using, as a parameter, a ratio between the maximum value and the minimum value determined at the first step. For example, inter predictor **218** performs $64 \times \text{min}/\text{max}$ as a division operation and calculates an integer close to the calculated result to derive the first value. Note that as an alternative way to the division operation, inter predictor **218** may use binary search to derive the first value. For example, assuming that "max"=100 and "min"=64, the first value can be derived to be 40 after 6 rounds of search in the range of [0, 64].

Subsequently, at the third step of the deriving process, inter predictor **218** can determine the second value and the third value based on the first value derived at the second step using the look up table shown in FIG. **18**. For example, inter predictor **218** can determine second value=434 and third value=271 based on the first value=40 derived at the second step using the look up table shown in FIG. **18**. Note that the value of the look up table shown in FIG. **18** is generated based on the assumption that the precision of the motion vector is 16, i.e., a motion distance of 1 pixel is 16. The value is also generated based on the assumption that the range of the first value is from 0 to 64. Accordingly, when the precision of the motion vector is different or the range of the first value is different, the value of the look up table shown in FIG. **18** may be different.

Subsequently, at the fourth step of the deriving process, inter predictor **218** parses a magnitude index indicating the magnitude of the delta motion vector, and determines a shift value using the table shown in FIG. **19**. Inter predictor **218** replaces, using the determined shift value, coordinates indicated by the second value and the third value obtained at the third step with the xy coordinates in (b) of FIG. **17**. For example, inter predictor **218** can replace coordinates indicated by the second value and the third value with the xy coordinates in (b) of FIG. **17** by performing a shift operation defined by equation (3) using the determined shift value to update the second value and the third value obtained at the third step.

MATH. 3

$$\text{Output} = (\text{input} + (1 \ll (\text{shift} - 1))) \gg \text{shift}; \text{ when } \text{shift} > 0 \quad (3)$$

input; when shift = 0

For example, it is assumed that the magnitude index parsed by inter predictor **218** is 1 (magnitude index=1) and the shift value is 6 from FIG. **19**. It is also assumed that at the third step, second value=434 and third value=271 are obtained. In this case, inter predictor **218** can perform the shift operation defined by equation (3) to update the second value to $(434+32) \gg 6 = 7$ and the third value to $(271+32) \gg 6 = 4$.

25

Subsequently, at the fifth step of the deriving process, when $\max = \text{abs}(m)$ is true, inter predictor **218** updates the ratio between the second value and the third value (second value/third value) to be of the same sign as the ratio between the component values of the first motion vector (x value/y value). In this case, inter predictor **218** can determine the coordinates indicating the first direction to be [second value, third value]. On the other hands, when $\max = \text{abs}(m)$ is not true, inter predictor **218** updates the ratio between the second value and the third value (second value/third value) to be of the same sign as the inverse ratio between the component values of the first motion vector (y value/x value). In this case, inter predictor **218** can determine the coordinates indicating the first direction to be [third value, second value].

In the example of the first step through the fourth step, the component values of the first motion vector are [100, 64] and $\max = \text{abs}(100)$, and thus inter predictor **218** can determine the coordinates indicating the first direction to be [7, 4]. Accordingly, the coordinates indicating the second direction are [-4, 7].

Note that at step **S10015**, inter predictor **218** may determine either one of the first direction or the second direction and either one of the positive side or the negative side as the one direction indicating the direction of the delta motion vector and obtain the magnitude in the determined one direction by parsing the magnitude index and the direction index obtained from the prediction parameter or the like. For example, when inter predictor **218** parses magnitude index=1 and direction index=0 from the prediction parameter or the like, the one direction indicating the direction of the delta motion vector is the first direction and it is denoted by [7, 4].

Aspect 4 of Inter Prediction

In aspect 3, the method for deriving, as the first direction, a direction substantially the same as the direction of the first motion vector using the direction of the first motion vector has been described, but the present disclosure is not limited to this. A direction closest to the direction of the first motion vector among predetermined multiple directions may be derived as the first direction. The following describes this case as aspect 4.

FIG. **20** is a flow chart illustrating an example of the internal processing of decoder **200** according to aspect 4 of Embodiment 1. FIG. **21** is for illustrating a delta motion vector used in inter prediction according to aspect 4 of Embodiment 1.

As shown in FIG. **20**, inter predictor **218** in decoder **200** derives, as the first direction among the first direction and the second direction which are perpendicular to each other, a direction closest to the direction of the first motion vector by selecting from among predetermined multiple directions (**S100141**).

Here, this step is specifically described with reference to FIG. **21**.

In FIG. **21**, as the predetermined multiple directions, the first direction, the second direction, the third direction, and the fourth direction are indicated. Note that the first direction includes not only the right diagonal direction in the first quadrant (+45°) but also a direction extending in the third quadrant (+225°). Similarly, the second direction includes not only the left diagonal direction in the fourth quadrant (-45°) but also a direction extending in the second quadrant (-225°). The third direction is indicated as a horizontal direction including the positive side and the negative side,

26

and the fourth direction is indicated as a vertical direction including the positive side and the negative side.

Inter predictor **218** derives the first direction and the second direction by selecting from among the predetermined multiple directions as shown in FIG. **21**.

More specifically, first, it is assumed that inter predictor **218** selects the first motion vector as shown in (a) of FIG. **21**. Subsequently, inter predictor **218** determines which one of the predetermined multiple directions is close to the direction of the first motion vector as shown in (a) of FIG. **21**. For example, inter predictor **218** may determine whether the ratio (m/n) between the component values of the first motion vector as shown in (a) of FIG. **21** is within a predetermined range (+45°+22.5°) including the first direction shown in (b) of FIG. **21**. In other words, inter predictor **218** can determine whether to be closest to the first direction by determining whether $(32/13) > (m/n) > (32/77)$ is true.

Subsequently, when the above relationship is satisfied, inter predictor **218** determines that the direction of the first motion vector is closest to the first direction. In other words, inter predictor **218** derives, as the first direction, a direction closest to the direction of the first motion vector by selecting from among the predetermined multiple directions.

In this way, inter predictor **218** can derive the first direction and the second direction which are perpendicular to each other, using the direction of the first motion vector. With this, inter predictor **218** can derive the first direction and the second direction which are perpendicular to each other, by selecting from among the predetermined multiple directions.

Note that step **S100141** corresponds to another example of the above deriving process of the first direction and the second direction at step **S10014**. The description of processing at step **S10015** and the following steps is omitted.

Aspect 5 of Inter Prediction

In the present aspect, inter predictor **218** in decoder **200** performs a process for directly selecting one direction from among predetermined multiple directions, to parse a delta motion vector. The following mainly describes the differences from aspect 1 through aspect 4.

FIG. **22** is a flow chart illustrating an example of the internal processing of decoder **200** according to aspect 5 of Embodiment 1. FIG. **23** is for illustrating a delta motion vector used in inter prediction according to aspect 5 of Embodiment 1.

Firstly, as shown in FIG. **22**, inter predictor **218** in decoder **200** parses a delta motion vector representing (i) one direction selected from among the predetermined multiple directions including diagonal directions and (ii) a distance from a base motion vector (**S2001**).

In the present aspect, firstly, inter predictor **218** parses information obtained from an encoded bitstream to determine the base motion vector. Subsequently, inter predictor **218** evaluates all the predetermined multiple directions along with their magnitudes (steps) to select a motion vector in one direction.

Here, this step is specifically described with reference to FIG. **23**. In FIG. **23**, as the predetermined multiple directions, the first direction through the eighth direction including diagonal directions are indicated. In the example shown in FIG. **23**, the first direction, the second direction, the third direction, and the fourth direction can be denoted by [0, 1], [1, 1], [1, 0], and [1, -1], respectively. Similarly, the fifth

direction, the sixth direction, the seventh direction, and the eighth direction can be denoted by [0, -1], [-1, -1], [-1, 0], and [-1, 1], respectively.

Inter predictor **218** may select at least the one direction using a flag included in the obtained prediction parameter. In other words, inter predictor **218** may select the one direction by parsing a motion direction syntax indicating the selected one direction and included in the obtained prediction parameter. Inter predictor **218** also may further select, from a predetermined list, the magnitude in one direction of the motion vector serving as the delta motion vector, by parsing a magnitude index included in the obtained prediction parameter. Here, the predetermined list is a list indicating a relationship between the magnitude index and the magnitude in the selected one direction of the motion vector, and may be a table as shown in FIG. 19, for example.

Next, as shown in FIG. 22, inter predictor **218** decodes the current block using at least the delta motion vector parsed at step S2001 (S2002). More specifically, inter predictor **218** decodes the current block, i.e., generates a prediction signal of the current block, by performing motion compensation using the delta motion vector parsed at step S2001 and the base motion vector.

Advantageous Effects of Aspect 1 Through Aspect

5

According to aspect 1 through aspect 5, in inter prediction, motion compensation can be performed using a motion vector having higher accuracy than a base motion vector by introducing a process for deriving a motion vector into inter prediction according to the present disclosure. With this, it is possible to improve encoding efficiency of inter prediction.

More specifically, according to aspect 1 through aspect 5, in inter prediction, the decoder or the like determines (selects) a base motion vector and parses a delta motion vector representing a difference from the base motion vector using a direction and a step. For example, in inter prediction in merge mode, the decoder or the like determines the base motion vector by selecting one MV predictor from a MV predictor list. Then, the delta motion vector representing a difference from the one MV predictor using the direction and the step is parsed using an encoded parameter or the like.

In this way, the decoder or the like can obtain the delta motion vector which is extension information of the base motion vector such as the one MV predictor and difference information between the base motion vector and the motion vector having higher accuracy than the base motion vector.

Then, the decoder or the like generates a prediction signal of a current block (decodes a current block) by performing motion compensation using the base motion vector and the delta motion vector obtained by the parsing.

In this way, the decoder derives, in inter prediction, a higher-accuracy motion vector. With this, it is possible to improve encoding efficiency of inter prediction.

As described above, according to aspect 1 through aspect 5, in inter prediction, a higher-accuracy motion vector can be derived. Accordingly, it is possible to achieve an encoder, a decoder, an encoding method, and a decoding method which can improve encoding efficiency of inter prediction.

Note that all the components described in aspect 1 are not necessarily required, and only some of the components in aspect 1 may be included.

Implementation Example of Encoder

FIG. 24 is a block diagram illustrating an implementation example of encoder **100** according to Embodiment 1.

Encoder **100** includes circuitry **160** and memory **162**. For example, the components in encoder **100** shown in FIG. 1 is implemented as circuitry **160** and memory **162** shown in FIG. 24.

Circuitry **160** performs information processing, and is accessible to memory **162**. For example, circuitry **160** is a dedicated or general-purpose electronic circuit for encoding a video. Circuitry **160** may be a processor such as a CPU. Circuitry **160** also may be an assembly of electronic circuits. Furthermore, for example, circuitry **160** may serve as components other than components for storing information among the components in encoder **100** shown in FIG. 1, etc.

Memory **162** is a dedicated or general-purpose memory that stores information for encoding a video in circuitry **160**. Memory **162** may be an electronic circuit, and be connected to circuitry **160**. Memory **162** also may be included in circuitry **160**. Memory **162** also may be an assembly of electronic circuits. Memory **162** also may be a magnetic disk, an optical disk, etc., and be referred to as a storage, a recording medium, etc. Memory **162** also may be a non-volatile memory or a volatile memory.

For example, memory **162** may store a video to be encoded, or a bitstream corresponding to the encoded video. Memory **162** also may store a program for encoding a video in circuitry **160**.

Furthermore, for example, memory **162** may serve as components for storing information among the components in encoder **100** shown in FIG. 1, etc. In particular, memory **162** may serve as block memory **118** and frame memory **122** shown in FIG. 1. More specifically, memory **162** may store a reconstructed block, a reconstructed picture, etc.

Note that in encoder **100**, all the components shown in FIG. 1, etc., need not be implemented, or all the foregoing processes need not be performed. Some of the components shown in FIG. 1, etc., may be included in another device, or some of the foregoing processes may be performed by another device. Then, in encoder **100**, some of the components shown in FIG. 1, etc., are implemented and some of the foregoing processes are performed, and thereby encoding efficiency is improved.

The following describes an operation example of encoder **100** shown in FIG. 24. In the operation example described below, FIG. 25 is a flow chart illustrating the operation example of encoder **100** shown in FIG. 24. For example, in encoding a video, encoder **100** shown in FIG. 24 performs the operation shown in FIG. 25.

In particular, using memory **162**, circuitry **160** in encoder **100** performs the processing as follows. In other words, firstly, in inter prediction for a current block, circuitry **160** determines a base motion vector (S311), and writes, in an encoded signal, a delta motion vector representing (i) one direction among predetermined directions including a diagonal direction and (ii) a distance from the base motion vector (S312). Next, circuitry **160** encodes the current block using the delta motion vector obtained at step S312 and the base motion vector (S313).

With this, encoder **100** derives, in inter prediction, a higher-accuracy motion vector, and thus it is possible to improve encoding efficiency of inter prediction. Accordingly, encoder **100** can improve encoding efficiency.

Implementation Example of Decoder

FIG. 26 is a block diagram illustrating an implementation example of decoder **200** according to Embodiment 1. Decoder **200** includes circuitry **260** and memory **262**. For

29

example, the components in decoder 200 shown in FIG. 10 are implemented as circuitry 260 and memory 262 shown in FIG. 26.

Circuitry 260 performs information processing, and is accessible to memory 262. For example, circuitry 260 is a dedicated or general-purpose electronic circuit for decoding a video. Circuitry 260 may be a processor such as a CPU. Circuitry 260 also may be an assembly of electronic circuits. Furthermore, for example, circuitry 260 may serve as components other than components for storing information among the components in decoder 200 shown in FIG. 10, etc.

Memory 262 is a dedicated or general-purpose memory that stores information for decoding a video in circuitry 260. Memory 262 may be an electronic circuit, and be connected to circuitry 260. Memory 262 also may be included in circuitry 260. Memory 262 also may be an assembly of electronic circuits. Memory 262 also may be a magnetic disk, an optical disk, etc., and be referred to as a storage, a recording medium, etc. Memory 262 also may be a non-volatile memory or a volatile memory.

For example, memory 262 may store a bitstream corresponding to an encoded video, or a video corresponding to a decoded bitstream. Memory 262 also may store a program for decoding a video in circuitry 260.

Furthermore, for example, memory 262 may serve as components for storing information among the components in decoder 200 shown in FIG. 10, etc. In particular, memory 262 may serve as block memory 210 and frame memory 214 shown in FIG. 10. More specifically, memory 262 may store a reconstructed block, a reconstructed picture, etc.

Note that in decoder 200, all the components shown in FIG. 10, etc., need not be implemented, or all the foregoing processes need not be performed. Some of the components shown in FIG. 10, etc., may be included in another device, or some of the foregoing processes may be performed by another device. Then, in decoder 200, some of the components shown in FIG. 10, etc., are implemented and some of the foregoing processes are performed, and thereby the motion compensation is effectively performed.

The following describes an operation example of decoder 200 shown in FIG. 26. FIG. 27 is a flow chart illustrating an operation example of decoder 200 shown in FIG. 26. For example, in decoding a video, decoder 200 shown in FIG. 26 performs the operation shown in FIG. 27.

In particular, using memory 262, circuitry 260 in encoder 200 performs the processing as follows. In other words, firstly, in inter prediction for a current block, circuitry 260 determines a base motion vector (S411), and parses a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector (S412). Next, using memory 262, circuitry 260 decodes the current block using the delta motion vector parsed at step S412 and the base motion vector (S413).

In this way, decoder 200 derives, in inter prediction, a higher-accuracy motion vector. With this, it is possible to improve encoding efficiency of inter prediction. Accordingly, decoder 200 can improve encoding efficiency.

Supplementary

Furthermore, encoder 100 and decoder 200 according to the present embodiment may be used as an image encoder and an image decoder, or may be used as a video encoder or a video decoder, respectively. Alternatively, encoder 100 and

30

decoder 200 are applicable as an inter prediction device (an inter-picture prediction device).

In other words, encoder 100 and decoder 200 may correspond to only intra predictor (intra-picture predictor) 216 and inter predictor (inter-picture predictor) 218, respectively. The remaining components such as transformer 106, inverse transformer 206, etc., may be included in another device.

Furthermore, in the present embodiment, each component may be configured by a dedicated hardware, or may be implemented by executing a software program suitable for each component. Each component may be implemented by causing a program executor such as a CPU or a processor to read out and execute a software program stored on a recording medium such as a hard disk or a semiconductor memory.

In particular, encoder 100 and decoder 200 may each include processing circuitry and a storage which is electrically connected to the processing circuitry and is accessible from the processing circuitry. For example, the processing circuitry corresponds to circuitry 160 or 260, and the storage corresponds to memory 162 or 262.

The processing circuitry includes at least one of a dedicated hardware and a program executor, and performs processing using the storage. Furthermore, when the processing circuitry includes the program executor, the storage stores a software program to be executed by the program executor.

Here, a software for implementing encoder 100, decoder 200, etc., according to the present embodiment is a program as follows.

In other words, the program may cause a computer to execute an encoding method including: first, in inter prediction for a current block, determining a base motion vector, and second, writing, in an encoded signal, a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and encoding the current block using the delta motion vector and the base motion vector.

Alternatively, the program may cause a computer to execute a decoding method including: first, in inter prediction for a current block, determining a base motion vector, and second, parsing a delta motion vector representing (i) one direction among a plurality of directions including a diagonal direction and (ii) a distance from the base motion vector; and decoding the current block using the delta motion vector and the base motion vector.

Furthermore, as described above, each component may be a circuit. The circuits may be integrated into a single circuit as a whole, or may be separated from each other. Furthermore, each component may be implemented as a general-purpose processor, or as a dedicated processor.

Furthermore, a process performed by a specific component may be performed by another component. Furthermore, the order of processes may be changed, or multiple processes may be performed in parallel. Furthermore, an encoding and decoding device may include encoder 100 and decoder 200.

The ordinal numbers used in the illustration such as first and second may be renumbered as needed. Furthermore, the ordinal number may be newly assigned to a component, etc., or may be deleted from a component, etc.

As described above, the aspects of encoder 100 and decoder 200 have been described based on the embodiment, but the aspects of encoder 100 and decoder 200 are not limited to this embodiment. Various modifications to the embodiment that can be conceived by those skilled in the art,

and forms configured by combining components in different embodiments without departing from the spirit of the present invention may be included in the scope of the aspects of encoder 100 and decoder 200.

This aspect may be implemented in combination with one or more of the other aspects according to the present disclosure. In addition, part of the processes in the flowcharts, part of the constituent elements of the apparatuses, and part of the syntax described in this aspect may be implemented in combination with other aspects.

Embodiment 2

As described in each of the above embodiments, each functional block can typically be realized as an MPU and memory, for example. Moreover, processes performed by each of the functional blocks are typically realized by a program execution unit, such as a processor, reading and executing software (a program) recorded on a recording medium such as ROM. The software may be distributed via, for example, downloading, and may be recorded on a recording medium such as semiconductor memory and distributed. Note that each functional block can, of course, also be realized as hardware (dedicated circuit).

Moreover, the processing described in each of the embodiments may be realized via integrated processing using a single apparatus (system), and, alternatively, may be realized via decentralized processing using a plurality of apparatuses. Moreover, the processor that executes the above-described program may be a single processor or a plurality of processors. In other words, integrated processing may be performed, and, alternatively, decentralized processing may be performed.

Embodiments of the present disclosure are not limited to the above exemplary embodiments; various modifications may be made to the exemplary embodiments, the results of which are also included within the scope of the embodiments of the present disclosure.

Next, application examples of the moving picture encoding method (image encoding method) and the moving picture decoding method (image decoding method) described in each of the above embodiments and a system that employs the same will be described. The system is characterized as including an image encoder that employs the image encoding method, an image decoder that employs the image decoding method, and an image encoder/decoder that includes both the image encoder and the image decoder. Other configurations included in the system may be modified on a case-by-case basis.

Usage Examples

FIG. 28 illustrates an overall configuration of content providing system ex100 for implementing a content distribution service. The area in which the communication service is provided is divided into cells of desired sizes, and base stations ex106, ex107, ex108, ex109, and ex110, which are fixed wireless stations, are located in respective cells.

In content providing system ex100, devices including computer ex111, gaming device ex112, camera ex113, home appliance ex114, and smartphone ex115 are connected to internet ex101 via internet service provider ex102 or communications network ex104 and base stations ex106 through ex110. Content providing system ex100 may combine and connect any combination of the above elements. The devices may be directly or indirectly connected together via a telephone network or near field communication rather than

via base stations ex106 through ex110, which are fixed wireless stations. Moreover, streaming server ex103 is connected to devices including computer ex111, gaming device ex112, camera ex113, home appliance ex114, and smartphone ex115 via, for example, internet ex101. Streaming server ex103 is also connected to, for example, a terminal in a hotspot in airplane ex117 via satellite ex116.

Note that instead of base stations ex106 through ex110, wireless access points or hotspots may be used. Streaming server ex103 may be connected to communications network ex104 directly instead of via internet ex101 or internet service provider ex102, and may be connected to airplane ex117 directly instead of via satellite ex116.

Camera ex113 is a device capable of capturing still images and video, such as a digital camera. Smartphone ex115 is a smartphone device, cellular phone, or personal handyphone system (PHS) phone that can operate under the mobile communications system standards of the typical 2G, 3G, 3.9G, and 4G systems, as well as the next-generation 5G system.

Home appliance ex118 is, for example, a refrigerator or a device included in a home fuel cell cogeneration system.

In content providing system ex100, a terminal including an image and/or video capturing function is capable of, for example, live streaming by connecting to streaming server ex103 via, for example, base station ex106. When live streaming, a terminal (e.g., computer ex111, gaming device ex112, camera ex113, home appliance ex114, smartphone ex115, or airplane ex117) performs the encoding processing described in the above embodiments on still-image or video content captured by a user via the terminal, multiplexes video data obtained via the encoding and audio data obtained by encoding audio corresponding to the video, and transmits the obtained data to streaming server ex103. In other words, the terminal functions as the image encoder according to one aspect of the present disclosure.

Streaming server ex103 streams transmitted content data to clients that request the stream. Client examples include computer ex111, gaming device ex112, camera ex113, home appliance ex114, smartphone ex115, and terminals inside airplane ex117, which are capable of decoding the above-described encoded data. Devices that receive the streamed data decode and reproduce the received data. In other words, the devices each function as the image decoder according to one aspect of the present disclosure.

Decentralized Processing

Streaming server ex103 may be realized as a plurality of servers or computers between which tasks such as the processing, recording, and streaming of data are divided. For example, streaming server ex103 may be realized as a content delivery network (CDN) that streams content via a network connecting multiple edge servers located throughout the world. In a CDN, an edge server physically near the client is dynamically assigned to the client. Content is cached and streamed to the edge server to reduce load times. In the event of, for example, some kind of an error or a change in connectivity due to, for example, a spike in traffic, it is possible to stream data stably at high speeds since it is possible to avoid affected parts of the network by, for example, dividing the processing between a plurality of edge servers or switching the streaming duties to a different edge server, and continuing streaming.

Decentralization is not limited to just the division of processing for streaming; the encoding of the captured data may be divided between and performed by the terminals, on

the server side, or both. In one example, in typical encoding, the processing is performed in two loops. The first loop is for detecting how complicated the image is on a frame-by-frame or scene-by-scene basis, or detecting the encoding load. The second loop is for processing that maintains image quality and improves encoding efficiency. For example, it is possible to reduce the processing load of the terminals and improve the quality and encoding efficiency of the content by having the terminals perform the first loop of the encoding and having the server side that received the content perform the second loop of the encoding. In such a case, upon receipt of a decoding request, it is possible for the encoded data resulting from the first loop performed by one terminal to be received and reproduced on another terminal in approximately real time. This makes it possible to realize smooth, real-time streaming.

In another example, camera ex113 or the like extracts a feature amount from an image, compresses data related to the feature amount as metadata, and transmits the compressed metadata to a server. For example, the server determines the significance of an object based on the feature amount and changes the quantization accuracy accordingly to perform compression suitable for the meaning of the image. Feature amount data is particularly effective in improving the precision and efficiency of motion vector prediction during the second compression pass performed by the server. Moreover, encoding that has a relatively low processing load, such as variable length coding (VLC), may be handled by the terminal, and encoding that has a relatively high processing load, such as context-adaptive binary arithmetic coding (CABAC), may be handled by the server.

In yet another example, there are instances in which a plurality of videos of approximately the same scene is captured by a plurality of terminals in, for example, a stadium, shopping mall, or factory. In such a case, for example, the encoding may be decentralized by dividing processing tasks between the plurality of terminals that captured the videos and, if necessary, other terminals that did not capture the videos and the server, on a per-unit basis. The units may be, for example, groups of pictures (GOP), pictures, or tiles resulting from dividing a picture. This makes it possible to reduce load times and achieve streaming that is closer to real-time.

Moreover, since the videos are of approximately the same scene, management and/or instruction may be carried out by the server so that the videos captured by the terminals can be cross-referenced. Moreover, the server may receive encoded data from the terminals, change reference relationship between items of data or correct or replace pictures themselves, and then perform the encoding. This makes it possible to generate a stream with increased quality and efficiency for the individual items of data.

Moreover, the server may stream video data after performing transcoding to convert the encoding format of the video data. For example, the server may convert the encoding format from MPEG to VP, and may convert H.264 to H.265.

In this way, encoding can be performed by a terminal or one or more servers. Accordingly, although the device that performs the encoding is referred to as a "server" or "terminal" in the following description, some or all of the processes performed by the server may be performed by the terminal, and likewise some or all of the processes performed by the terminal may be performed by the server. This also applies to decoding processes.

3D, Multi-Angle

In recent years, usage of images or videos combined from images or videos of different scenes concurrently captured

or the same scene captured from different angles by a plurality of terminals such as camera ex113 and/or smart-phone ex115 has increased. Videos captured by the terminals are combined based on, for example, the separately-obtained relative positional relationship between the terminals, or regions in a video having matching feature points.

In addition to the encoding of two-dimensional moving pictures, the server may encode a still image based on scene analysis of a moving picture either automatically or at a point in time specified by the user, and transmit the encoded still image to a reception terminal. Furthermore, when the server can obtain the relative positional relationship between the video capturing terminals, in addition to two-dimensional moving pictures, the server can generate three-dimensional geometry of a scene based on video of the same scene captured from different angles. Note that the server may separately encode three-dimensional data generated from, for example, a point cloud, and may, based on a result of recognizing or tracking a person or object using three-dimensional data, select or reconstruct and generate a video to be transmitted to a reception terminal from videos captured by a plurality of terminals.

This allows the user to enjoy a scene by freely selecting videos corresponding to the video capturing terminals, and allows the user to enjoy the content obtained by extracting, from three-dimensional data reconstructed from a plurality of images or videos, a video from a selected viewpoint. Furthermore, similar to with video, sound may be recorded from relatively different angles, and the server may multiplex, with the video, audio from a specific angle or space in accordance with the video, and transmit the result.

In recent years, content that is a composite of the real world and a virtual world, such as virtual reality (VR) and augmented reality (AR) content, has also become popular. In the case of VR images, the server may create images from the viewpoints of both the left and right eyes and perform encoding that tolerates reference between the two viewpoint images, such as multi-view coding (MVC), and, alternatively, may encode the images as separate streams without referencing. When the images are decoded as separate streams, the streams may be synchronized when reproduced so as to recreate a virtual three-dimensional space in accordance with the viewpoint of the user.

In the case of AR images, the server superimposes virtual object information existing in a virtual space onto camera information representing a real-world space, based on a three-dimensional position or movement from the perspective of the user. The decoder may obtain or store virtual object information and three-dimensional data, generate two-dimensional images based on movement from the perspective of the user, and then generate superimposed data by seamlessly connecting the images. Alternatively, the decoder may transmit, to the server, motion from the perspective of the user in addition to a request for virtual object information, and the server may generate superimposed data based on three-dimensional data stored in the server in accordance with the received motion, and encode and stream the generated superimposed data to the decoder. Note that superimposed data includes, in addition to RGB values, an α value indicating transparency, and the server sets the α value for sections other than the object generated from three-dimensional data to, for example, 0, and may perform the encoding while those sections are transparent. Alternatively, the server may set the background to a predetermined RGB value, such as a chroma key, and generate data in which areas other than the object are set as the background.

Decoding of similarly streamed data may be performed by the client (i.e., the terminals), on the server side, or divided therebetween. In one example, one terminal may transmit a reception request to a server, the requested content may be received and decoded by another terminal, and a decoded signal may be transmitted to a device having a display. It is possible to reproduce high image quality data by decentralizing processing and appropriately selecting content regardless of the processing ability of the communications terminal itself. In yet another example, while a TV, for example, is receiving image data that is large in size, a region of a picture, such as a tile obtained by dividing the picture, may be decoded and displayed on a personal terminal or terminals of a viewer or viewers of the TV. This makes it possible for the viewers to share a big-picture view as well as for each viewer to check his or her assigned area or inspect a region in further detail up close.

In the future, both indoors and outdoors, in situations in which a plurality of wireless connections are possible over near, mid, and far distances, it is expected to be able to seamlessly receive content even when switching to data appropriate for the current connection, using a streaming system standard such as MPEG-DASH. With this, the user can switch between data in real time while freely selecting a decoder or display apparatus including not only his or her own terminal, but also, for example, displays disposed indoors or outdoors. Moreover, based on, for example, information on the position of the user, decoding can be performed while switching which terminal handles decoding and which terminal handles the displaying of content. This makes it possible to, while in route to a destination, display, on the wall of a nearby building in which a device capable of displaying content is embedded or on part of the ground, map information while on the move. Moreover, it is also possible to switch the bit rate of the received data based on the accessibility to the encoded data on a network, such as when encoded data is cached on a server quickly accessible from the reception terminal or when encoded data is copied to an edge server in a content delivery service.

Scalable Encoding

The switching of content will be described with reference to a scalable stream, illustrated in FIG. 29, that is compression coded via implementation of the moving picture encoding method described in the above embodiments. The server may have a configuration in which content is switched while making use of the temporal and/or spatial scalability of a stream, which is achieved by division into and encoding of layers, as illustrated in FIG. 29. Note that there may be a plurality of individual streams that are of the same content but different quality. In other words, by determining which layer to decode up to based on internal factors, such as the processing ability on the decoder side, and external factors, such as communication bandwidth, the decoder side can freely switch between low resolution content and high resolution content while decoding. For example, in a case in which the user wants to continue watching, at home on a device such as a TV connected to the internet, a video that he or she had been previously watching on smartphone ex115 while on the move, the device can simply decode the same stream up to a different layer, which reduces server side load.

Furthermore, in addition to the configuration described above in which scalability is achieved as a result of the pictures being encoded per layer and the enhancement layer is above the base layer, the enhancement layer may include

metadata based on, for example, statistical information on the image, and the decoder side may generate high image quality content by performing super-resolution imaging on a picture in the base layer based on the metadata. Super-resolution imaging may be improving the SN ratio while maintaining resolution and/or increasing resolution. Metadata includes information for identifying a linear or a non-linear filter coefficient used in super-resolution processing, or information identifying a parameter value in filter processing, machine learning, or least squares method used in super-resolution processing.

Alternatively, a configuration in which a picture is divided into, for example, tiles in accordance with the meaning of, for example, an object in the image, and on the decoder side, only a partial region is decoded by selecting a tile to decode, is also acceptable. Moreover, by storing an attribute about the object (person, car, ball, etc.) and a position of the object in the video (coordinates in identical images) as metadata, the decoder side can identify the position of a desired object based on the metadata and determine which tile or tiles include that object. For example, as illustrated in FIG. 30, metadata is stored using a data storage structure different from pixel data such as an SEI message in HEVC. This metadata indicates, for example, the position, size, or color of the main object.

Moreover, metadata may be stored in units of a plurality of pictures, such as stream, sequence, or random access units. With this, the decoder side can obtain, for example, the time at which a specific person appears in the video, and by fitting that with picture unit information, can identify a picture in which the object is present and the position of the object in the picture.

Web Page Optimization

FIG. 31 illustrates an example of a display screen of a web page on, for example, computer ex111. FIG. 32 illustrates an example of a display screen of a web page on, for example, smartphone ex115. As illustrated in FIG. 31 and FIG. 32, a web page may include a plurality of image links which are links to image content, and the appearance of the web page differs depending on the device used to view the web page. When a plurality of image links are viewable on the screen, until the user explicitly selects an image link, or until the image link is in the approximate center of the screen or the entire image link fits in the screen, the display apparatus (decoder) displays, as the image links, still images included in the content or I pictures, displays video such as an animated gif using a plurality of still images or I pictures, for example, or receives only the base layer and decodes and displays the video.

When an image link is selected by the user, the display apparatus decodes giving the highest priority to the base layer. Note that if there is information in the HTML code of the web page indicating that the content is scalable, the display apparatus may decode up to the enhancement layer. Moreover, in order to guarantee real time reproduction, before a selection is made or when the bandwidth is severely limited, the display apparatus can reduce delay between the point in time at which the leading picture is decoded and the point in time at which the decoded picture is displayed (that is, the delay between the start of the decoding of the content to the displaying of the content) by decoding and displaying only forward reference pictures (I picture, P picture, forward reference B picture). Moreover, the display apparatus may purposely ignore the reference relationship between pictures and coarsely decode all B and P pictures as forward refer-

ence pictures, and then perform normal decoding as the number of pictures received over time increases.

Autonomous Driving

When transmitting and receiving still image or video data such two- or three-dimensional map information for autonomous driving or assisted driving of an automobile, the reception terminal may receive, in addition to image data belonging to one or more layers, information on, for example, the weather or road construction as metadata, and associate the metadata with the image data upon decoding. Note that metadata may be assigned per layer and, alternatively, may simply be multiplexed with the image data.

In such a case, since the automobile, drone, airplane, etc., including the reception terminal is mobile, the reception terminal can seamlessly receive and decode while switching between base stations among base stations ex106 through ex110 by transmitting information indicating the position of the reception terminal upon reception request. Moreover, in accordance with the selection made by the user, the situation of the user, or the bandwidth of the connection, the reception terminal can dynamically select to what extent the metadata is received or to what extent the map information, for example, is updated.

With this, in content providing system ex100, the client can receive, decode, and reproduce, in real time, encoded information transmitted by the user.

Streaming of Individual Content

In content providing system ex100, in addition to high image quality, long content distributed by a video distribution entity, unicast or multicast streaming of low image quality, short content from an individual is also possible. Moreover, such content from individuals is likely to further increase in popularity. The server may first perform editing processing on the content before the encoding processing in order to refine the individual content. This may be achieved with, for example, the following configuration.

In real-time while capturing video or image content or after the content has been captured and accumulated, the server performs recognition processing based on the raw or encoded data, such as capture error processing, scene search processing, meaning analysis, and/or object detection processing. Then, based on the result of the recognition processing, the server—either when prompted or automatically—edits the content, examples of which include: correction such as focus and/or motion blur correction; removing low-priority scenes such as scenes that are low in brightness compared to other pictures or out of focus; object edge adjustment; and color tone adjustment. The server encodes the edited data based on the result of the editing. It is known that excessively long videos tend to receive fewer views. Accordingly, in order to keep the content within a specific length that scales with the length of the original video, the server may, in addition to the low-priority scenes described above, automatically clip out scenes with low movement based on an image processing result. Alternatively, the server may generate and encode a video digest based on a result of an analysis of the meaning of a scene.

Note that there are instances in which individual content may include content that infringes a copyright, moral right, portrait rights, etc. Such an instance may lead to an unfavorable situation for the creator, such as when content is shared beyond the scope intended by the creator. Accordingly, before encoding, the server may, for example, edit

images so as to blur faces of people in the periphery of the screen or blur the inside of a house, for example. Moreover, the server may be configured to recognize the faces of people other than a registered person in images to be encoded, and when such faces appear in an image, for example, apply a mosaic filter to the face of the person. Alternatively, as pre- or post-processing for encoding, the user may specify, for copyright reasons, a region of an image including a person or a region of the background be processed, and the server may process the specified region by, for example, replacing the region with a different image or blurring the region. If the region includes a person, the person may be tracked in the moving picture, and the head region may be replaced with another image as the person moves.

Moreover, since there is a demand for real-time viewing of content produced by individuals, which tends to be small in data size, the decoder first receives the base layer as the highest priority and performs decoding and reproduction, although this may differ depending on bandwidth. When the content is reproduced two or more times, such as when the decoder receives the enhancement layer during decoding and reproduction of the base layer and loops the reproduction, the decoder may reproduce a high image quality video including the enhancement layer. If the stream is encoded using such scalable encoding, the video may be low quality when in an unselected state or at the start of the video, but it can offer an experience in which the image quality of the stream progressively increases in an intelligent manner. This is not limited to just scalable encoding; the same experience can be offered by configuring a single stream from a low quality stream reproduced for the first time and a second stream encoded using the first stream as a reference.

Other Usage Examples

The encoding and decoding may be performed by LSI ex500, which is typically included in each terminal. LSI ex500 may be configured of a single chip or a plurality of chips. Software for encoding and decoding moving pictures may be integrated into some type of a recording medium (such as a CD-ROM, a flexible disk, or a hard disk) that is readable by, for example, computer ex111, and the encoding and decoding may be performed using the software. Furthermore, when smartphone ex115 is equipped with a camera, the video data obtained by the camera may be transmitted. In this case, the video data is coded by LSI ex500 included in smartphone ex115.

Note that LSI ex500 may be configured to download and activate an application. In such a case, the terminal first determines whether it is compatible with the scheme used to encode the content or whether it is capable of executing a specific service. When the terminal is not compatible with the encoding scheme of the content or when the terminal is not capable of executing a specific service, the terminal first downloads a codec or application software then obtains and reproduces the content.

Aside from the example of content providing system ex100 that uses internet ex101, at least the moving picture encoder (image encoder) or the moving picture decoder (image decoder) described in the above embodiments may be implemented in a digital broadcasting system. The same encoding processing and decoding processing may be applied to transmit and receive broadcast radio waves superimposed with multiplexed audio and video data using, for

example, a satellite, even though this is geared toward multicast whereas unicast is easier with content providing system ex100.

Hardware Configuration

FIG. 33 illustrates smartphone ex115. FIG. 34 illustrates a configuration example of smartphone ex115. Smartphone ex115 includes antenna ex450 for transmitting and receiving radio waves to and from base station ex110, camera ex465 capable of capturing video and still images, and display ex458 that displays decoded data, such as video captured by camera ex465 and video received by antenna ex450. Smartphone ex115 further includes user interface ex466 such as a touch panel, audio output unit ex457 such as a speaker for outputting speech or other audio, audio input unit ex456 such as a microphone for audio input, memory ex467 capable of storing decoded data such as captured video or still images, recorded audio, received video or still images, and mail, as well as decoded data, and slot ex464 which is an interface for SIM ex468 for authorizing access to a network and various data. Note that external memory may be used instead of memory ex467.

Moreover, main controller ex460 which comprehensively controls display ex458 and user interface ex466, power supply circuit ex461, user interface input controller ex462, video signal processor ex455, camera interface ex463, display controller ex459, modulator/demodulator ex452, multiplexer/demultiplexer ex453, audio signal processor ex454, slot ex464, and memory ex467 are connected via bus ex470.

When the user turns the power button of power supply circuit ex461 on, smartphone ex115 is powered on into an operable state by each component being supplied with power from a battery pack.

Smartphone ex115 performs processing for, for example, calling and data transmission, based on control performed by main controller ex460, which includes a CPU, ROM, and RAM. When making calls, an audio signal recorded by audio input unit ex456 is converted into a digital audio signal by audio signal processor ex454, and this is applied with spread spectrum processing by modulator/demodulator ex452 and digital-analog conversion and frequency conversion processing by transmitter/receiver ex451, and then transmitted via antenna ex450. The received data is amplified, frequency converted, and analog-digital converted, inverse spread spectrum processed by modulator/demodulator ex452, converted into an analog audio signal by audio signal processor ex454, and then output from audio output unit ex457. In data transmission mode, text, still-image, or video data is transmitted by main controller ex460 via user interface input controller ex462 as a result of operation of, for example, user interface ex466 of the main body, and similar transmission and reception processing is performed. In data transmission mode, when sending a video, still image, or video and audio, video signal processor ex455 compression encodes, via the moving picture encoding method described in the above embodiments, a video signal stored in memory ex467 or a video signal input from camera ex465, and transmits the encoded video data to multiplexer/demultiplexer ex453. Moreover, audio signal processor ex454 encodes an audio signal recorded by audio input unit ex456 while camera ex465 is capturing, for example, a video or still image, and transmits the encoded audio data to multiplexer/demultiplexer ex453. Multiplexer/demultiplexer ex453 multiplexes the encoded video data and encoded audio data using a predetermined scheme, modulates and converts the data using modulator/demodulator

(modulator/demodulator circuit) ex452 and transmitter/receiver ex451, and transmits the result via antenna ex450.

When video appended in an email or a chat, or a video linked from a web page, for example, is received, in order to decode the multiplexed data received via antenna ex450, multiplexer/demultiplexer ex453 demultiplexes the multiplexed data to divide the multiplexed data into a bitstream of video data and a bitstream of audio data, supplies the encoded video data to video signal processor ex455 via synchronous bus ex470, and supplies the encoded audio data to audio signal processor ex454 via synchronous bus ex470. Video signal processor ex455 decodes the video signal using a moving picture decoding method corresponding to the moving picture encoding method described in the above embodiments, and video or a still image included in the linked moving picture file is displayed on display ex458 via display controller ex459. Moreover, audio signal processor ex454 decodes the audio signal and outputs audio from audio output unit ex457. Note that since real-time streaming is becoming more and more popular, there are instances in which reproduction of the audio may be socially inappropriate depending on the user's environment. Accordingly, as an initial value, a configuration in which only video data is reproduced, i.e., the audio signal is not reproduced, is preferable. Audio may be synchronized and reproduced only when an input, such as when the user clicks video data, is received.

Although smartphone ex115 was used in the above example, three implementations are conceivable: a transceiver terminal including both an encoder and a decoder; a transmitter terminal including only an encoder; and a receiver terminal including only a decoder. Further, in the description of the digital broadcasting system, an example is given in which multiplexed data obtained as a result of video data being multiplexed with, for example, audio data, is received or transmitted, but the multiplexed data may be video data multiplexed with data other than audio data, such as text data related to the video. Moreover, the video data itself rather than multiplexed data maybe received or transmitted.

Although main controller ex460 including a CPU is described as controlling the encoding or decoding processes, terminals often include GPUs. Accordingly, a configuration is acceptable in which a large area is processed at once by making use of the performance ability of the GPU via memory shared by the CPU and GPU or memory including an address that is managed so as to allow common usage by the CPU and GPU. This makes it possible to shorten encoding time, maintain the real-time nature of the stream, and reduce delay. In particular, processing relating to motion estimation, deblocking filtering, sample adaptive offset (SAO), and transformation/quantization can be effectively carried out by the GPU instead of the CPU in units of, for example pictures, all at once.

Although only some exemplary embodiments of the present disclosure have been described in detail above, those skilled in the art will readily appreciate that many modifications are possible in the exemplary embodiments without materially departing from the novel teachings and advantages of the present disclosure. Accordingly, all such modifications are intended to be included within the scope of the present disclosure.

What is claimed is:

1. An encoder, comprising:
circuitry; and
memory, wherein

41

using the memory, the circuitry:

selects a base motion vector from candidate motion vectors;

selects a direction of a delta motion vector from four predetermined directions which differ from each other, the four predetermined directions including (i) a first direction, (ii) a second direction perpendicular to the first direction, (iii) a third direction perpendicular to the second direction, and (iv) a fourth direction perpendicular to the third direction;

determines a magnitude of the delta motion vector, the magnitude depending on precision of the delta motion vector;

writes an index into a bitstream, the index indicating the magnitude; and

modifies the base motion vector using the delta motion vector to encode a current block.

42

2. A decoder, comprising:

circuitry; and

memory, wherein

using the memory, the circuitry:

selects a base motion vector from candidate motion vectors;

selects a direction of a delta motion vector from four predetermined directions which differ from each other, the four predetermined directions including (i) a first direction, (ii) a second direction perpendicular to the first direction, (iii) a third direction perpendicular to the second direction, and (iv) a fourth direction perpendicular to the third direction;

determines a magnitude of the delta motion vector according to (i) an index stored in a bitstream and (ii) precision of the delta motion vector; and

modifies the base motion vector using the delta motion vector to decode a current block.

* * * * *